



A Modular Vehicle Presence Detection System Using BLE, LoRa and MQTT

SETU Higher Diploma in Computer Science
Aidan Clancy
20062567

28/3/26

Word Count: 7,782

Repo Link <https://github.com/A-Clancy/GateMateLab/releases/tag/v1.0>

Declaration of Authenticity

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks' for shorter excerpt and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student: 

Date: 28/03/26

Dedication

Though she won't ever read it, this is dedicated to my wife for continuing to indulge my follies.

Abstract

GateMate is a privacy-first vehicle presence detection system designed for gates and access points. The system uses ultra-sonic distance-based sensing combined with time-threshold detection logic to generate a stable presence state, avoiding reliance on cameras or cloud-based image processing. A modular architecture is employed, separating sensing, transport, and processing through an MQTT messaging layer. The system supports both Wi-Fi and LoRa communication, enabling operation across domestic, rural, and temporary site environments where range and infrastructure constraints vary. By abstracting transport from detection logic, GateMate provides a flexible and extensible platform for real-world deployment while maintaining a low-power, local-first design.

Contents

1.	Introduction	1
2.	Research.....	1
2.1.	Requirements Analysis	1
2.2.	User Stories.....	3
2.3.	Market and Existing Solution Analysis	4
2.4.	Repo Analysis	5
2.5.	Technical Research and Selection	5
3.	Methodology	8
4.	Modelling	9
4.1.	System Model.....	9
4.2.	Logical Components.....	9
4.3.	MVP Scope and Future Extensions	10
4.4.	Data Flow	10
4.5.	State Model	11
5.	Development	12
6.	Sprint 1 - Git Init and BLE Hardware Integration	13
6.1.	Pre-Sprint Objectives	13
6.2.	Implementation and Technical Approach.....	13
6.3.	Evidence and Validation	13
6.4.	Outcome	14
6.5.	Limitations and Issues Encountered	14
6.6.	Implications for next Iteration.....	14
7.	Sprint 2 – Presence Detection Logic.....	15
7.1.	Pre-Sprint Objectives	15
7.2.	Implementation and Technical Approach.....	15
7.3.	Evidence and Validation	17
7.4.	Outcome	17
7.5.	Limitations and Issues Encountered	17
7.6.	Implications for next Iteration.....	18
8.	Sprint 3 – Notifications, including Dashboards and MQTT Decoupling.....	19
8.1.	Pre-Sprint Objectives	19
8.2.	Implementation and Technical Approach.....	20
8.3.	Evidence and Validation	25

8.4.	Outcome	26
8.5.	Limitations and Issues Encountered	26
8.6.	Implications for next Iteration	26
9.	Sprint 4 – Hardware	27
9.1.	Pre-Sprint Objectives	27
9.2.	Implementation and Technical Approach.....	27
9.3.	Evidence and Validation	31
9.4.	Outcome	32
9.5.	Limitations and Issues Encountered	32
9.6.	Implications for next Iteration	32
10.	Sprint 5A – Backlog Work.....	33
10.1.	Low Battery Notification.....	33
10.2.	Separation of concerns	34
10.3.	Visual Notification	34
10.4.	Outcome	35
11.	Sprint 5B - LoRa (Stretch Goal)	36
11.1.	Pre-Sprint Objectives	36
11.2.	Implementation and Technical Approach.....	36
11.3.	Evidence and Validation	37
11.4.	Outcomes	39
11.5.	Limitations and Issues Encountered	39
11.6.	Implications for next iteration	39
12.	Sprint 6 - Refinement	40
12.1.	Pre-Sprint Objectives	40
12.2.	Implementation and Technical Approach.....	40
12.3.	Approach Taken	41
12.4.	Evidence and Validation	41
12.5.	Outcome	42
13.	Reflections.....	43
13.1.	What I Learned	43
13.2.	What I Achieved.....	43
13.3.	Problems Encountered and Solutions.....	44
14.	Conclusion.....	44
15.	Bibliography	45

16.	Appendices	46
16.1.	Appendix – Trello Boards	46
16.2.	Appendix - Glossary of Terms	47
16.3.	Appendix – Use of AI.....	48

Table of Figures

Figure 1 HA Add-ons, base case.....	7
Figure 2 GateMate System Model (MVP)	9
Figure 3 GateMate Data Flow	10
Figure 4 State Diagram	11
Figure 5 Sprint 1 Objectives (Trello)	13
Figure 6 Sprint 2 Objectives (Trello)	15
Figure 7 Presence detection pipeline showing sensor input, filtering, presence abstraction and downstream consumers.....	16
Figure 8 Code Extract Showing Dwell Time Filter.	16
Figure 9 Verified sensor readings captured over a two-hour period	17
Figure 10 Initially unexplained oscillations in sensor readings	18
Figure 11 Pre-Sprint Objectives (Trello).....	19
Figure 12 Code Extract showing Notification Acknowledgement mechanism.....	20
Figure 13 Basic Dashboard, Version 1.0	21
Figure 14 Diagnostic Dashboard, V1.0.....	21
Figure 15 Code Extract Showing MQTT Publishing	22
Figure 16 Built-in HA MQTT Listener demonstrating presence detection	22
Figure 17 MQTT Client subscribed to GateMate presence alerts	23
Figure 18 Event driven architecture demonstrating MQTT decoupling between detection logic and downstream consumers	23
Figure 19 Code Extract Showing a portion of the python subscriber script.....	24
Figure 20 Phone notification on Ntfy app while connected to 5G only.	24
Figure 21 Alerts on a wearable device, via Ntfy app and MQTT	25
Figure 22 Presence Detection Helper History, toggling between states	25
Figure 23 Presence Detection Toggled Off	26
Figure 24 Presence Detection Toggled On.....	26
Figure 25 Sprint 4 Objectives (Trello)	27
Figure 26 ESP32 Bluetooth Proxy, showing in ESPHome, within HA.....	28
Figure 27 Built in Bluetooth adapter & new proxy both visible in HA.	28
Figure 28 BLE Direct Transport Path	28
Figure 29 Enclosure adapted to take Ultrasonic Sensor.....	29
Figure 30 ESP32 Bluetooth Proxy in compact housing	29
Figure 31 GateMate hardware in sealed housing.....	30
Figure 32 Hardware architecture of the portable sensing node	31
Figure 33 Backlog Work, Sprint 5 (Trello).....	33
Figure 34 Code Extract Showing a part of the low battery notification	34
Figure 35 Refactoring Config file to improve readability etc	34
Figure 36 GateMate system architecture showing sensing, transport, processing, and integration layers	35
Figure 37 LoRa relay transport path.....	36
Figure 38 Authoritative sensor abstraction layer	37
Figure 38 LoRa Boards sending and receiving information.....	38
Figure 40 Updated hardware including LoRa antenna	38
Figure 41 Sprint 6 Objectives (Trello)	40
Figure 42 UI Improvements to clearly convey the relevant information.....	41

Figure 43 Diagnostic Dashboard showing different readings when LoRa is unavailable (LoRa reading is old) 42

1. Introduction

This report documents the project from the RAMP phase through to final project delivery. The following pages detail the research and development work through a series of sprints.

2. Research

This section documents the research and analysis carried out during the RAMP phase of the GateMate project. It begins by 1) establishing user, functional, and environmental requirements, then 2) examines existing commercial and open-source solutions, and 3) concludes with an evaluation of candidate technologies. The purpose of this section is to narrow the technical and design space and to inform subsequent modelling and development decisions.

2.1. Requirements Analysis

The various requirements are analysed below and overleaf, considering 1) User requirements, 2) Core Functionality, 3) UX requirements, and 4) environmental and operational requirements. This is then summarised in a Requirements Analysis Matrix in 2.1.5.

2.1.1. User Requirements

The GateMate system is designed to detect the presence of vehicles at a domestic/rural/remote access point and notify users accordingly. The system may evolve over time to support multiple user types, each with distinct needs.

The identified user categories are as follows:

1. The primary user will be the research practitioner
2. Secondary users: household members, with different alert/interaction preferences
3. Tertiary users: Household members without the Homes Assistant (**HA**) app

2.1.2. Core Functional Requirements

The core functionality is well defined:

1. Detect the presence of a vehicle at the gate and notify the user.
2. Differentiate between brief stops (e.g. delivery driver) and longer dwell times (e.g. waiting to be let in).
3. Notification methods may include:
 - a. HA mobile notification.
 - b. MQTT message to local dashboards.
 - c. Alexa-based announcement or hardware indicator.
4. The system should minimise false positives.
5. The system should be built in a modular fashion to allow for hardware upgrades and additional features.
6. The system should be capable of being adapted in the future to become a long-range, low-power, presence detection system.

2.1.3. User Interaction Requirements

The main interface will be a HA dashboard.

The project will explore additional hardware-based notifications (e.g. LED indicator). This would also need to include a method for user acknowledgment (e.g. physical button).

2.1.4. Environmental and Operational Requirements

The detection system must operate outdoors. Power and Wi-Fi are available in the vicinity of the test environment. In the later revisions, neither power nor Wi-Fi may be available. Hardware must be weather-resistant or housed in a waterproof enclosure. The system must be mountable.

2.1.5. Requirements Analysis Matrix

To prioritise features and requirements for this project, a structured categorisation approach aligned with the MoSCoW prioritisation method was utilised. The MoSCoW framework divides requirements into “Must have”, “Should have”, “Could have”, and “Won’t have this time” (Agile Business Consortium, 2022). This should provide a sound basis for distinguishing core requirements from enhancements and lower-priority ideas. This technique is widely used in agile and project management environments to help with project scoping.

The initial feature analysis is shown in Table 1 overleaf. This will form the basis for the initial Minimum Viable Proposal (MVP) as well as future potential iterations. The Feature Analysis may well evolve over the duration of the project.

Table 1 Feature Analysis

Feature	Description	MoSCoW Priority
Vehicle presence detection	Detect objects within a defined range (e.g. 50–300 cm). Duration-based filtering (>5s) helps differentiate passers-by from visitors.	Must have
MQTT messaging	Publishes MQTT messages based on detection state (“Vehicle Detected”, “Vehicle no longer detected”).	Must have
Home Assistant notifications	Triggers mobile notifications based on detection events.	Must have
Weatherproof mounting	Enclosure or housing suitable for reliable outdoor operation.	Must have
Notifications outside of home network	Allow GateMate to notify users of presence detections when they are not at home	Should have
Sense HAT / LED style feedback	Displays alerts locally inside the house (e.g. on Sense HAT or LED panel or <u>some</u> other method).	Should have
User acknowledgement button	A hardware or dashboard-based method to dismiss or acknowledge alerts.	Should have
Bluetooth Wi-Fi Gateway	The system may require a gateway to transmit Bluetooth signals over longer distances via Wi-Fi	Could Have
ThingSpeak logging	Optional cloud logging for diagnostics and visualisation.	Could have
Web dashboard	Additional browser-based status view, separate from Home Assistant.	Could have
BLE sensor integration (Shelly Blu)	Use of Shelly Blu Distance for simplified detection logic. Requires research into BLE range and behaviour.	Could have
Camera-based image capture	Trigger ESP32-CAM to take and log images during detection events.	Won’t have (this time)
LoRa or offline comms fallback	Use of LoRa or ESP-NOW as a backup to Wi-Fi. Useful for remote gate use cases.	Won’t have (this time)
Low power consumption, long range comms	Use of LoRa, in a low power system with options for battery and solar power.	Won’t have (this time)

2.2. User Stories

User stories are informal, natural-language descriptions of a software feature from the perspective of the end user. They are used to describe goals without detailed specifications and help focus development on delivering value (Rehkopf, 2019)

Some user stories are considered in Table 2.

Table 2 User Stories

ID	User Type	User Story	Priority	Notes
US1	Primary	As a homeowner, I want to be notified when a vehicle waits at the gate for more than a few seconds, so that I know whether to open it.	Must have	Core functional use case
US2	Secondary	As a homeowner, I want to receive a push notification when a vehicle is detected, so that I am alerted regardless of where I am.	Must have	Requires MQTT and/or HA automation
US3	Tertiary	As a household member, I want a visual indicator in the house when someone is at the gate, so that I can respond quickly.	Should have	LED / Sense HAT message
US4	Tertiary	As a child with no phone, I want a visual/aural notification and acknowledgement if presence is detected.	Should have	LED / Sense HAT message / Alexa Announcement / HA Text to Speech (TTS)
US5	Tertiary	As a visitor without a phone, I want the system to notify someone inside without relying on mobile apps, so that I can still be let in.	Should have	Requires fallback interface (e.g. indoor indicator)
US6	Primary	As a homeowner, I want to be able to acknowledge an alert manually, so that it stops repeating or clears the state.	Should have	Physical or virtual button
US7	Developer	As a developer, I want to log detection data to ThingSpeak or similar, so that I can review historical performance.	Could have	Useful for debugging / insight
US8	Primary	As a tech-savvy user, I want to access a web dashboard, so that I can see gate activity in a browser.	Could have	Parallel to Home Assistant UI
US9	Developer	As a developer, I want to add image capture for detections, so that I can visually confirm events in future iterations.	Won't have (this time)	Subject to BLE testing
US10	Developer	As a rural homeowner, I want to explore LoRa as a fallback comms method, so that I can use the system at off-grid sites.	Won't have (this time)	Stretch goal

2.3. Market and Existing Solution Analysis

This market analysis evaluates existing solutions against the long-term technical objectives of the GateMate project, i.e. to detect presence using a low-power, battery- or solar-powered device, and communicate over a long range without reliance on local Wi-Fi.

GateMate is initially a proof-of-concept device based on consumer technologies, but with potential ultimate applications in agricultural and industrial settings.

For this project, the market analysis is expanded to include an evaluation of existing commercial products and technical approaches that address the same problem. A constraints-driven analysis was carried out on existing solutions across these sectors, and the various solutions were categorised by their underlying technological approach. The findings are summarised in Table 3 below.

Table 3 Constraint Driven Analysis of available solutions

Tech	Detection Method	Power Usage	Connection Type	Comms Range	Privacy Implications	Project Fit
PIR motion sensor	Passive motion detection	Very low power	Typically hard-wired, Wi-Fi versions also available with higher power demands	Short	No personal data captured	Simple, but limited discrimination
Ultrasonic sensor	Active distance measurement	Moderate power draw; suitable for battery with optimisation	Wi-Fi required	Short	No personal data captured	Strong for MVP; flexible and controllable
Wi-Fi camera e.g. Ring	Motion + video capture	High power	Strong Wi-Fi required	Medium	High (image capture)	Unsuitable for remote locations
4G/LTE camera, e.g. Reolink Go Series	Motion + video capture	High power. Battery + solar is available.	Ongoing mobile data subscription	Long	High (image capture)	Effective but high cost and complexity

Market analysis did not show any products meeting the exact fit. This could be indicative of a niche in the market, or perhaps, an entire lack of a market. Mercifully, the business case is beyond the scope of this project.

2.4. Repo Analysis

As part of the research phase, a review of existing open-source projects was undertaken to understand how similar problems have been addressed in practice. The objective was to identify common architectural patterns, limitations, and design decisions relevant to the GateMate project. The repositories reviewed broadly fall into three categories, considered below.

2.4.1. ESP32-based ultrasonic distance sensing

Ultrasonic distance sensing using ESP32 microcontrollers is widely demonstrated across small repositories and tutorial-backed examples. A commonly referenced and maintained example set is provided by Random Nerd Tutorials, with accompanying source code hosted on GitHub can be found here - <https://github.com/RuiSantosdotme/ESP32-Course>

This repo demonstrates timing-based distance measurement using ESP32 and includes useful information on hardware wiring and signal handling. As it is a tutorial, the examples tend to be standalone. It does not cover power-optimisation or deployment. GateMate will extend these concepts by introducing dwell-time logic, event abstraction, and system integration rather than raw distance reporting.

2.4.2. MQTT-based presence reporting

The HA Core repository (<https://github.com/home-assistant/core>) contains extensive implementations of MQTT-based sensors and binary sensors, which are directly relevant to GateMate's integration layer.

There are examples of event-driven state updates and it demonstrates clear separation between sensor producers and consumers and solid automation and notification mechanisms. These concepts were integrated into the GateMate development. It does not address sensor, power and connectivity constraints.

2.4.3. Long-range low-power sensor nodes.

The HelTec Automation repo (https://github.com/HelTecAutomation/Heltec_ESP32) provides current reference implementations for ESP32 boards with integrated LoRa radios, including examples designed for low-power and event-based communication. This includes Long-range, low-bandwidth communication, support for deep sleep and battery operation. It is considerably more complex than the other repos, which is appropriate given its application to the stretch goal element of the project. During the implementation phase, these HelTec boards were selected for the LoRa implementation.

2.5. Technical Research and Selection

This section evaluates candidate sensing technologies suitable for detecting presence. The analysis focuses on the constraints identified earlier, particularly outdoor deployment, power consumption, communication range, and system architecture. The objective is to determine an appropriate technology for the GateMate MVP while maintaining a credible path toward future low-power, long-range operation.

The options are considered in Table 4. It is clear from the table that ultrasonic sensing remains the most appropriate detection method. The Shelly Blu Distance offers the best balance

between sensing capability, deployment robustness, and power efficiency, with BLE reception identified as the primary technical risk to be validated.

The outcomes of this technical evaluation directly inform the system architecture, data flow, and state models presented in the following section.

Table 4 Technology Comparison and Selection Matrix

Technology	Sensing Method	Installation	Power Usage	Comms Method and Protocols	Key Constraints	Assessment for GateMate
ESP32 + Ultrasonic (DIY)	Ultrasonic distance measurement	Requires enclosure, mounting, weatherproofing	Moderate (active sensing + Wi-Fi)	Wi-Fi (MQTT)	Power consumption, enclosure reliability	Strong for MVP validation and logic development
Shelly Blu Distance	Ultrasonic distance measurement (self-contained)	Purpose-built, weatherproof, compact	Low (battery-powered)	BLE	BLE range and reception reliability	Preferred candidate for deployment-focused iteration
PIR Sensor	Passive infrared motion detection	Simple outdoor mounting	Very low	Depends on controller	Cannot distinguish dwell time reliably	Poor fit for core requirement
Magnetic Vehicle Sensor	Magnetic field disturbance	Ground installation required	Very low	Varies	Vehicle-only detection, invasive install	Technically strong but impractical
Camera-based systems	Motion + image capture	Requires mounting, power, connectivity	High	Wi-Fi or 4G	Power, privacy, subscriptions	Conflicts with project objectives
LoRa-based node (future)	Event-based sensing	Custom build or specialist hardware	Very low	Sub-GHz / LoRa	Higher complexity, gateway required	Strong long-term architectural direction

2.5.1. Home Assistant

HA is a free, open-source, platform for home automation platform. It is built with a focus on privacy, and local network control. It has an active community who maintain well-documented resources for installation, operation and integration. HA acts as a central hub and runs on low-cost hardware, e.g. Raspberry Pi. It has a wide array of integrations that allows most of the most popular home automation products and services to be controlled and automated from HA dashboards. There are alternative home automation platforms available, such as openHAB or Homekit, but HA sits at the nexus of open-source, free and stable. These characteristics make HA a suitable integration platform for GateMate.

A brief description of the terminology used in HA is included as an appendix in 16.2. This may be a useful starting point for those less familiar with the platform.

During the RAMP phase, a small number of HA add-ons were evaluated in a standalone lab environment to assess their suitability for supporting version control, traceability, and reproducibility.

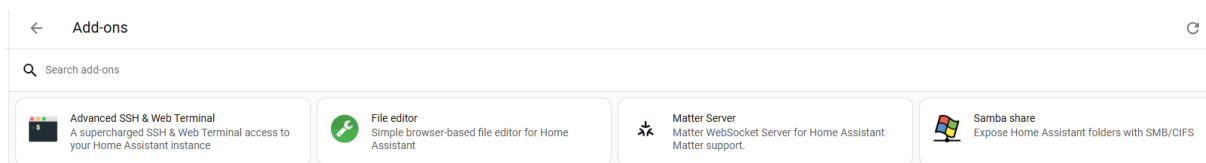


Figure 1 HA Add-ons, base case

2.5.2. Limitations

In the author's HA set up, notifications outside the home network are not enabled. This is available as a paid feature. This could potentially be circumvented with the use of an MQTT broker.

3. Methodology

This section outlines the development approach adopted for the GateMate project. It describes how implementation will proceed following the completion of the RAMP phase and explains how technical uncertainty associated with hardware integration and system behaviour will be managed during development.

This project adopts an iterative, incremental software engineering methodology, informed by Agile principles and structured around a clearly defined Minimum Viable Product (MVP). This approach was selected to reflect both the scale of the project and the technical uncertainty inherent in integrating physical sensing hardware with a software-driven home automation platform.

A linear or sequential lifecycle model (e.g. waterfall) was considered unsuitable, as key aspects of the system behaviour — including ultrasonic sensing reliability in an outdoor environment, dwell-time filtering, and wireless communication performance — cannot be fully specified or validated in advance. These elements require empirical testing and refinement through repeated implementation cycles.

Development is therefore organised into short, time-boxed sprints, each intended to either:

- a) Validate a specific technical assumption (e.g. sensing thresholds, BLE reception reliability), or
- b) Deliver a discrete unit of functionality aligned with the MVP definition.

User requirements identified during the RAMP phase are prioritised using the MoSCoW method, ensuring that core functionality is delivered first while preventing uncontrolled scope expansion. Informal user stories are used to translate these requirements into implementation tasks, supporting traceability between analysis, development activity, and delivered features.

To support controlled testing and reproducibility, a standalone HA environment is used for development and demonstration. This environment is logically and physically separated from any live home automation system, allowing safe experimentation, consistent testing conditions, and clear documentation without risk to production automations or personal data.

Planning and progress tracking are managed using Trello, with tasks organised by sprint. Source code and configuration artefacts are version-controlled using Git, with frequent commits used to document incremental progress, design decisions, and refinements made in response to testing outcomes.

4. Modelling

This section presents a set of lightweight models used to clarify system boundaries, component responsibilities, and expected system behaviour for the GateMate MVP. The models are intended to support implementation planning and reasoning about system behaviour rather than to act as fixed or exhaustive specifications.

4.1. System Model

This section presents a high-level system model for the GateMate MVP. The purpose of the model is to define system boundaries, identify core components, and clarify responsibility separation between sensing, processing, and user interaction. This is shown in Figure 2 and discussed in further detail in the following sections.

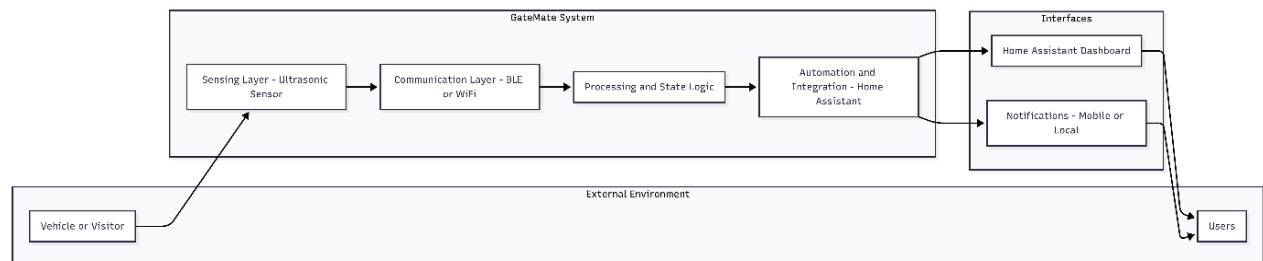


Figure 2 GateMate System Model (MVP)

4.1.1. System Boundary

The GateMate system is defined as the combination of sensing hardware, communication mechanisms, processing logic, and integrations required to detect presence at a domestic gate and notify users. Physical entities such as vehicles or people, as well as user devices (mobile phones, speakers, indicators), are considered external to the system.

Within the system boundary, GateMate is responsible for detecting presence, applying dwell-time logic, maintaining presence state, and triggering notifications through HA.

4.2. Logical Components

4.2.1. Sensing Layer

The sensing layer is responsible for measuring the distance between the gate and any object within range. The proposed hardware is a self-contained ultrasonic sensor (Shelly Blu Distance) communicating via Bluetooth Low Energy (BLE).

The sensing layer produces raw distance measurements only and does not perform any filtering or decision-making.

4.2.2. Communication Layer

The communication layer transports sensor data from the sensing hardware to the processing environment. BLE is used when integrating the Shelly Blu Distance via a Bluetooth proxy, which connects to HA via Wi-Fi. MQTT may be used for communication outside of the home network. This layer is responsible solely for reliable data delivery and does not interpret sensor values.

4.2.3. Processing and State Layer

The processing layer resides within HA and contains the core system logic. It applies dwell-time filtering to distance measurements to differentiate between brief pass-by events and sustained presence. The system maintains a small number of discrete presence states, such as presence detected, and presence cleared. Transient or noisy readings are filtered at this stage to minimise false positives.

4.2.4. Integration and Automation Layer

This layer consists of HA automations and integrations that respond to changes in presence state. Its responsibilities include triggering notifications, updating dashboards, and publishing MQTT messages where required. Automations are designed to be stateless where possible, relying on the presence state maintained by the processing layer.

4.2.5. User Interaction Layer

User interaction is primarily mediated through HA dashboards and notifications. Optional local indicators, such as LEDs, may be used to provide visual or audible alerts. Where implemented, acknowledgement mechanisms allow users to clear or silence notifications.

4.3. MVP Scope and Future Extensions

The MVP supports a single active sensing source at any given time, dwell-time-based presence detection, and notification delivery via HA. More advanced capabilities such as image capture, long-range communication (e.g. LoRa), power-optimised operation, or multi-sensor fusion are explicitly excluded from the MVP and are reserved for future iterations.

4.4. Data Flow

The GateMate data flow is illustrated in Figure 3 and considered below.

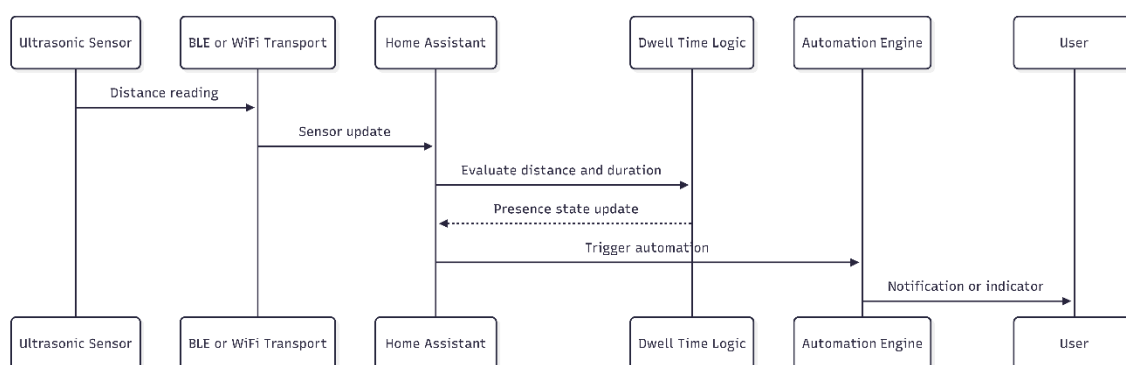


Figure 3 GateMate Data Flow

GateMate's data flow begins at the sensing layer, where an ultrasonic sensor produces periodic distance measurements representing the current clearance between the sensor and any object in its range.

Distance readings are sent to HA via the relevant communication layer. In all likelihood, the sensor will be out of reliable Bluetooth range, so a BLE to Wi-Fi proxy will be required.

Within HA, the raw distance stream is treated as an input signal rather than a decision. GateMate's core processing logic evaluates each reading against a configured distance window

and applies dwell-time filtering. A presence condition is only considered valid when readings remain within the threshold range for a defined period, reducing detections caused by passers-by, sensor noise, or brief stops. When the dwell-time condition is satisfied, the system transitions into a “presence detected” state. When readings no longer satisfy the threshold condition for a defined clearance period, the system transitions into a “presence cleared” state.

State transitions are the primary output of the processing layer. These state changes trigger HA automations, which deliver user-facing outcomes such as mobile notifications, dashboard updates, and optional local indicators. Where MQTT is used as an event transport or for local dashboards, GateMate publishes a small set of stable event messages (e.g. presence detected / presence cleared) rather than streaming raw distance values, ensuring that downstream consumers interact with a simple, consistent abstraction.

This design intentionally separates measurement, transport, interpretation, and notification. It allows the sensing hardware to be swapped without changing the higher-level automation behaviour, and it supports controlled MVP scope while leaving a clear path for future extensions such as long-range communications or additional notification channels.

4.5. State Model

GateMate uses a small state model to convert noisy distance readings into stable presence events suitable for HA automations. The model is intentionally minimal: it represents only the states required to support dwell-time filtering and reliable notifications in the MVP. It is illustrated in Figure 4 and considered further below.

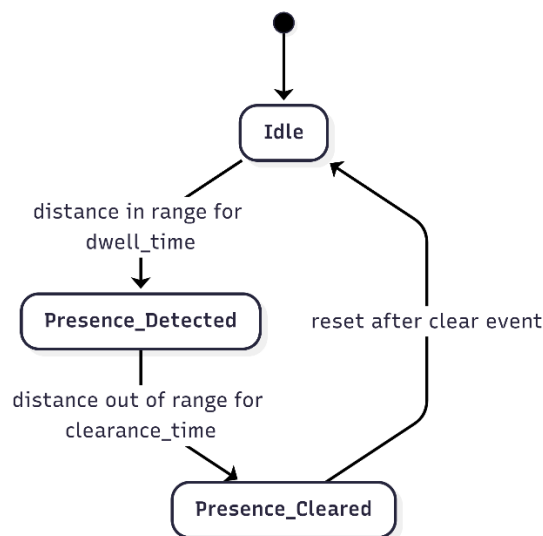


Figure 4 State Diagram

4.5.1. States

Idle

Default state. No valid presence condition is active.

Presence Detected

An object is considered present after distance readings remain within the configured threshold range for at least the dwell-time duration. This state represents a confirmed arrival or waiting event.

Presence Cleared

The object is no longer considered present after readings fall outside the configured threshold range for a defined clearance period. This state exists to ensure the “departure” event is explicit and not triggered by a single noisy measurement.

4.5.2. Transitions (conditions)

Idle → Presence Detected

Condition: distance is within the detection range continuously for $\geq$ dwell time.

Presence Detected → Presence Cleared

Condition: distance is outside the detection range continuously for $\geq$ clearance time.

Presence Cleared → Idle

Condition: clearance event has been emitted, and the system is ready to accept a new detection.

5. Development

The models developed in the previous section inform the implementation approach for GateMate. The system and state models guide the separation of concerns during development, while the data flow model informs event handling and automation design within the HA environment.

The development phase of the GateMate project is planned and managed using Trello. A Trello board has been created to organise development tasks into short, two-week sprints, with individual cards representing discrete units of work derived from the user stories and feature prioritisation completed during the RAMP phase. The pre-commencement plan is included in Appendix 0. The live Trello board may be viewed [here](#), but please note that this may be updated without notice.

The Trello board is structured around a small number of sprints, each with a clearly defined focus. Early sprints concentrate on implementing and validating the MVP functionality, including presence detection, dwell-time logic, and reliable event delivery into HA. Later sprints are reserved for refinement, integration improvements, and limited enhancement work where time permits.

The following sections track the development through each sprint in turn.

6. Sprint 1 - Git Init and BLE Hardware Integration

6.1. Pre-Sprint Objectives

This iteration focused on establishing the foundational development environment and validating initial hardware connectivity. The primary goals from the Trello plan are shown in Figure 5.

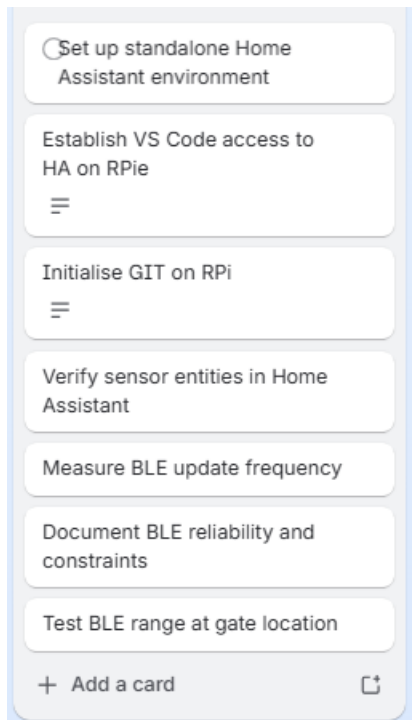


Figure 5 Sprint 1 Objectives (Trello)

6.2. Implementation and Technical Approach

A new HA environment was deployed and configured to support direct file access and Git-based version control. This enabled structured development and traceability of configuration changes throughout the project.

VS Code was used as the primary development environment, allowing rapid iteration while maintaining a clear commit history.

The Shelly Blu Distance sensor was paired with HA, creating sensor entities for distance and battery. These were verified within the HA entity registry and formed the basis for subsequent processing and automation logic.

6.3. Evidence and Validation

Sensor telemetry was successfully received within HA following pairing.

Intermittent zero-value readings were observed, indicating potential sensor or communication artefacts. This behaviour was documented for further investigation.

Testing at the intended gate location confirmed that the native Bluetooth adapter did not provide sufficient range, establishing the need for a Bluetooth proxy.

6.4. Outcome

A stable development environment was established, and the sensor was successfully integrated with HA.

This provided the foundation for implementing presence detection logic in subsequent sprints.

6.5. Limitations and Issues Encountered

Two key issues were identified:

- Intermittent zero-value readings from the sensor
- Insufficient Bluetooth range for the intended deployment location

6.6. Implications for next Iteration

The findings from this sprint directly informed the priorities of the subsequent iteration. In particular:

- Investigating and mitigating zero-value readings

Introducing a Bluetooth proxy to extend BLE coverage. These items were added to the development backlog and scheduled for investigation in later sprints.

7. Sprint 2 – Presence Detection Logic

Following the hardware validation work carried out in Sprint 2, Sprint 3 focused on implementing the core detection logic required to interpret sensor readings.

7.1. Pre-Sprint Objectives

The objectives for Sprint 2 are shown in Figure 6. These tasks were prioritised because they represent the core decision-making layer of the GateMate system. Until this logic was in place, downstream features such as notifications, MQTT publishing and dashboard indicators could not be implemented reliably.

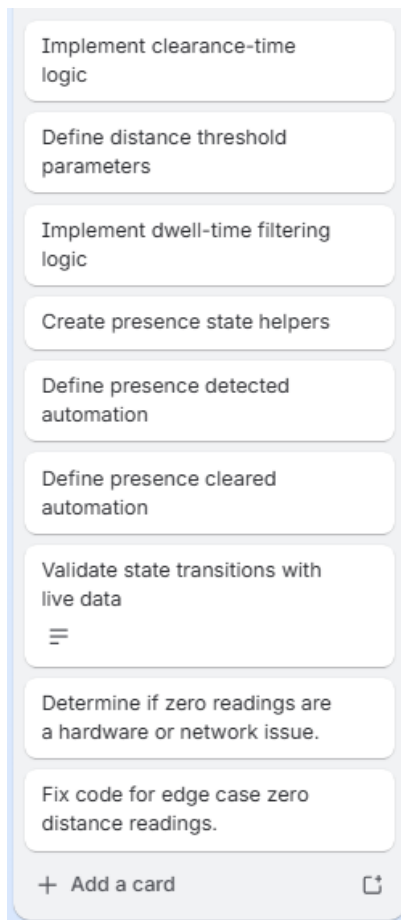


Figure 6 Sprint 2 Objectives (Trello)

7.2. Implementation and Technical Approach

7.2.1. Presence Detection Logic

This sprint focused on the design and calibration of the logic used to determine whether a vehicle was present at the gate. The processing pipeline used to convert raw sensor readings into an authoritative presence state is illustrated in Figure 7.

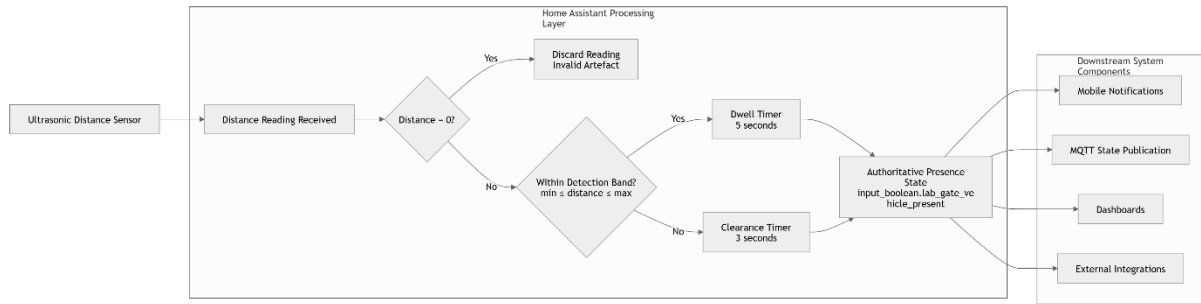


Figure 7 Presence detection pipeline showing sensor input, filtering, presence abstraction and downstream consumers

An authoritative presence state was introduced using a HA helper (input_boolean.lab_gate_vehicle_present), acting as the single source of truth for downstream systems. This avoided duplication of logic across notifications, dashboards, and integrations. This reduces duplicated logic and improves maintainability.

Vehicle presence was defined using a configurable distance band, allowing the detection zone to be tuned to the physical environment without modifying code. To improve robustness, dwell-time filtering was introduced. The system only transitions to a detected state when readings remain within the threshold range for a defined duration. A clearance-time filter was also applied to prevent oscillation during departure events. The associated code is shown in Figure 8.

```
config > automations > ! 10_detection.yaml > ...
1 - id: lab_gate_vehicle_detected_after_dwell
2   alias: Lab Gate Vehicle Detected After Dwell
3   description: Set vehicle present when distance remains within band for dwell time.
4   triggers:
5     - trigger: template
6       value_template: >-
7         {% set raw = states('sensor.lab_gate_distance_sensor_distance') %}
8         {% set d_mm = raw | float(none) %}
9         {% if d_mm is none or d_mm == 0 %}
10          false
11        {% else %}
12          {% set d = d_mm / 1000 %}
13          {% set min_m = states('input_number.lab_gate_distance_min_m') | float %}
14          {% set max_m = states('input_number.lab_gate_distance_max_m') | float %}
15          {{ d >= min_m and d <= max_m }}
16        {% endif %}
17      for:
18        seconds: 5
19    conditions:
20      - condition: state
21        entity_id: input_boolean.lab_gate_vehicle_present
22        state: "off"
23    actions:
24      - action: script.lab_gate_set_vehicle_present
25    mode: single
```

Figure 8 Code Extract Showing Dwell Time Filter.

7.2.2. Zero Reading Errors

To improve robustness, dwell-time filtering was introduced. The system only transitions to a detected state when readings remain within the threshold range for a defined duration. A clearance-time filter was also applied to prevent oscillation during departure events. The sprint concluded with validation using live sensor data. Multi-hour observations showed stable readings within expected physical bounds and confirmed that the system only changed state when the configured threshold and timing conditions were satisfied.

7.3. Evidence and Validation

Zero-value readings were identified as sensor artefacts and excluded from evaluation prior to threshold and timing logic. Figure 9 shows sensor readings remained stable following exclusion of zero-value artefacts. Presence state transitions occurred only when sustained threshold conditions were met, confirming correct operation of the dwell-time and clearance-time logic.

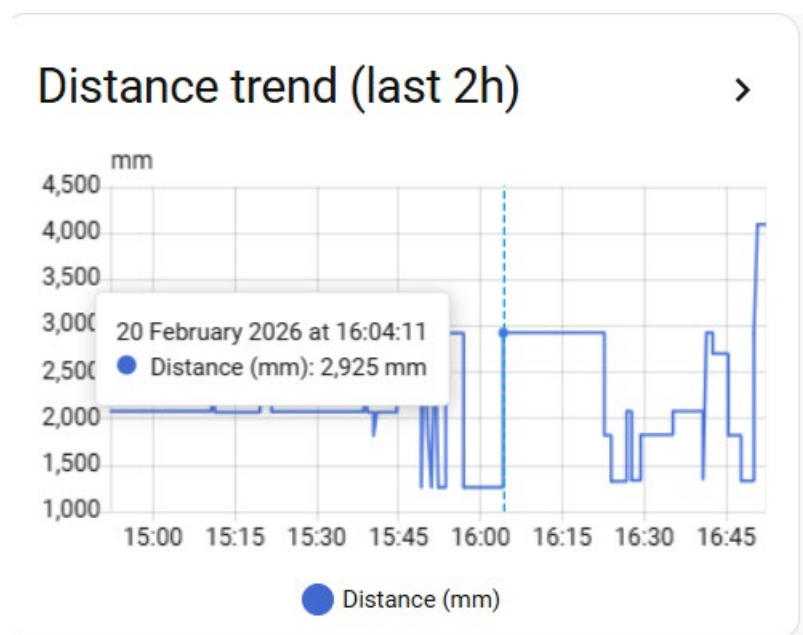


Figure 9 Verified sensor readings captured over a two-hour period

7.4. Outcome

Raw distance measurements were successfully converted into a stable binary presence abstraction. This established the system's core decision-making layer and enabled subsequent implementation of notifications, MQTT publishing, and dashboard integration.

7.5. Limitations and Issues Encountered

Two issues were identified:

- Inconsistent use of units (metres vs millimetres) introduced avoidable complexity during calibration.
- Apparent oscillations in sensor readings during unattended testing (Figure 10).

Further investigation confirmed that oscillation was actually caused by environmental factors – a 3rd party had inadvertently placed a curved reflective surfaces in the path of the sensor, affecting the ultrasonic beam rather than any logic instability.

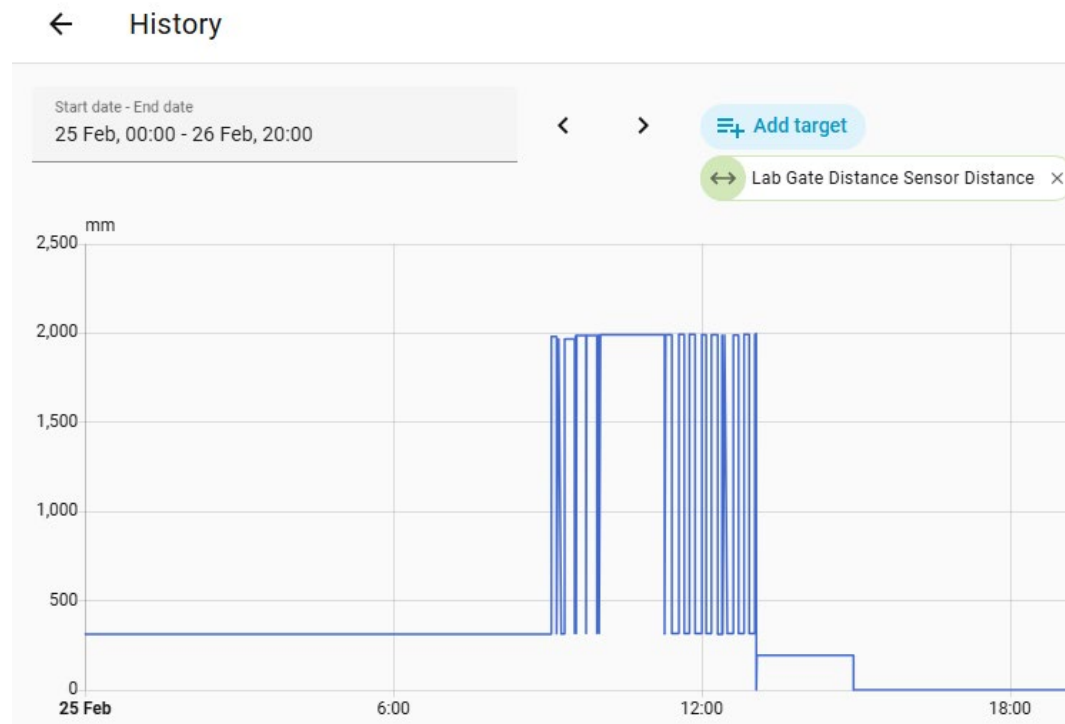


Figure 10 Initially unexplained oscillations in sensor readings

7.6. Implications for next Iteration

With stable detection logic in place, subsequent development could focus on user-facing outputs, including notifications, dashboards, and MQTT-based integration.

The sprint also highlighted the importance of controlled installation geometry and consistent unit handling during deployment.

8. Sprint 3 – Notifications, including Dashboards and MQTT Decoupling.

Following the implementation of reliable presence detection, this sprint focused on delivering system outputs and demonstrating architectural decoupling using MQTT.

This sprint also implemented basic dashboards and explored the stretch-goal of MQTT transport beyond the local HA network.

These tasks extended the system from a working detection prototype into a usable notification platform. Due to the breadth of work completed, this sprint could reasonably have been divided into multiple smaller iterations.

8.1. Pre-Sprint Objectives

The objectives for Sprint 3 are shown in Figure 11.

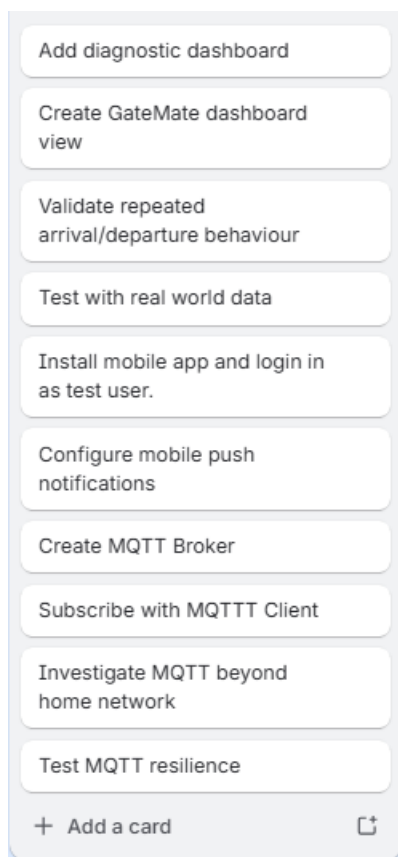


Figure 11 Pre-Sprint Objectives (Trello)

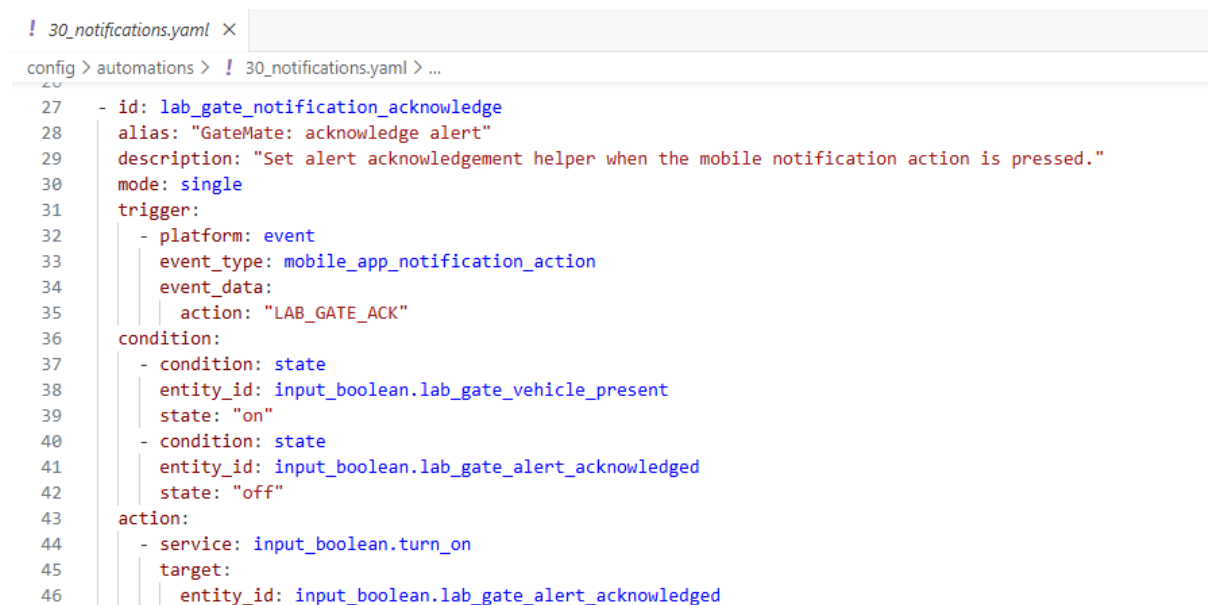
8.2. Implementation and Technical Approach

8.2.1. Notification Architecture

Notification logic was abstracted into reusable scripts to avoid duplication and maintain separation between detection logic and notification transport.

Helper entities were introduced to support throttling, deduplication and escalation behaviour. This ensured that repeated arrivals or boundary oscillation did not result in excessive notifications being sent to users.

An acknowledgement action was also implemented to allow users to suppress further escalation once a notification had been received. This provides a simple interaction mechanism while preserving the underlying presence state. This can be seen in Figure 12.



```
! 30_notifications.yaml ×
config > automations > ! 30_notifications.yaml > ...
27 - id: lab_gate_notification_acknowledge
28   alias: "GateMate: acknowledge alert"
29   description: "Set alert acknowledgement helper when the mobile notification action is pressed."
30   mode: single
31   trigger:
32     - platform: event
33       event_type: mobile_app_notification_action
34       event_data:
35         action: "LAB_GATE_ACK"
36   condition:
37     - condition: state
38       entity_id: input_boolean.lab_gate_vehicle_present
39       state: "on"
40     - condition: state
41       entity_id: input_boolean.lab_gate_alert_acknowledged
42       state: "off"
43   action:
44     - service: input_boolean.turn_on
45       target:
46         entity_id: input_boolean.lab_gate_alert_acknowledged
```

Figure 12 Code Extract showing Notification Acknowledgement mechanism

This approach allows additional notification transports to be added in future without modifying the core detection logic.

8.2.2. Dashboard Interfaces

Two lightweight dashboards were developed to support operational monitoring and validation during testing. The primary GateMate view, Figure 13, provides a high-level status display of the authoritative vehicle presence state, notification acknowledgement status, and key helper parameters, enabling rapid verification of system behaviour during live testing.

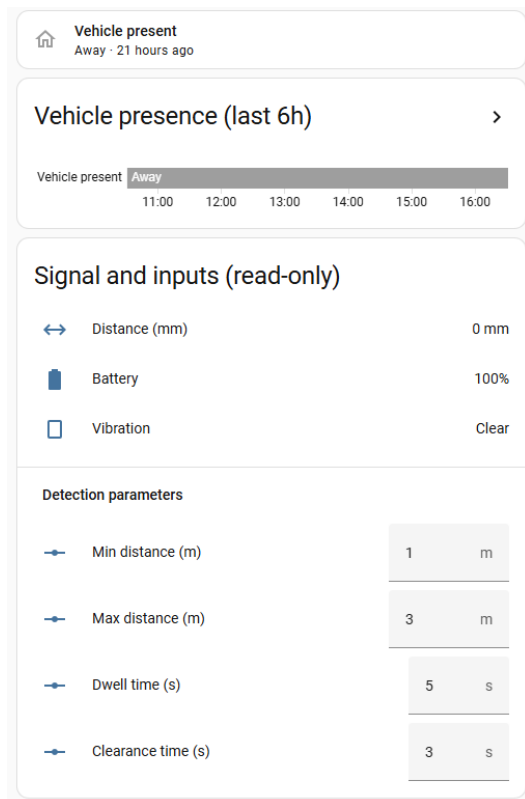


Figure 13 Basic Dashboard, Version 1.0

A secondary Diagnostics view, Figure 14, exposes raw distance readings, threshold values, and timing parameters, allowing direct observation of sensor stability and state transitions. The separation of operational and diagnostic views supports both end-user usability and engineering evaluation without introducing unnecessary interface complexity.

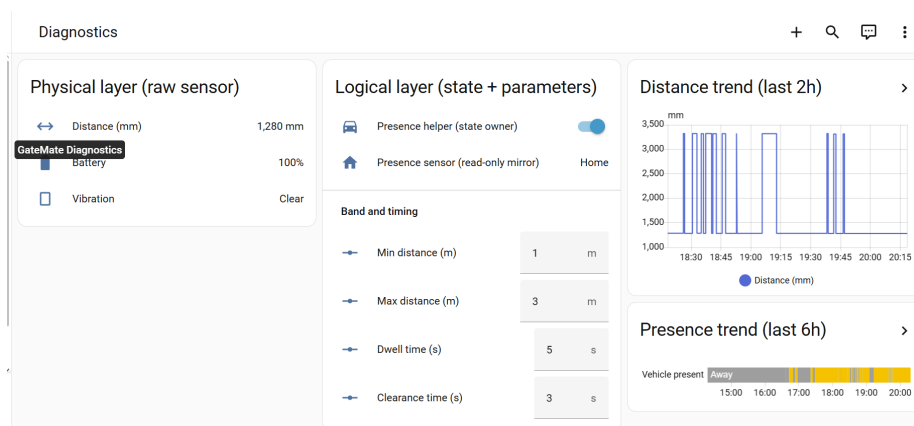
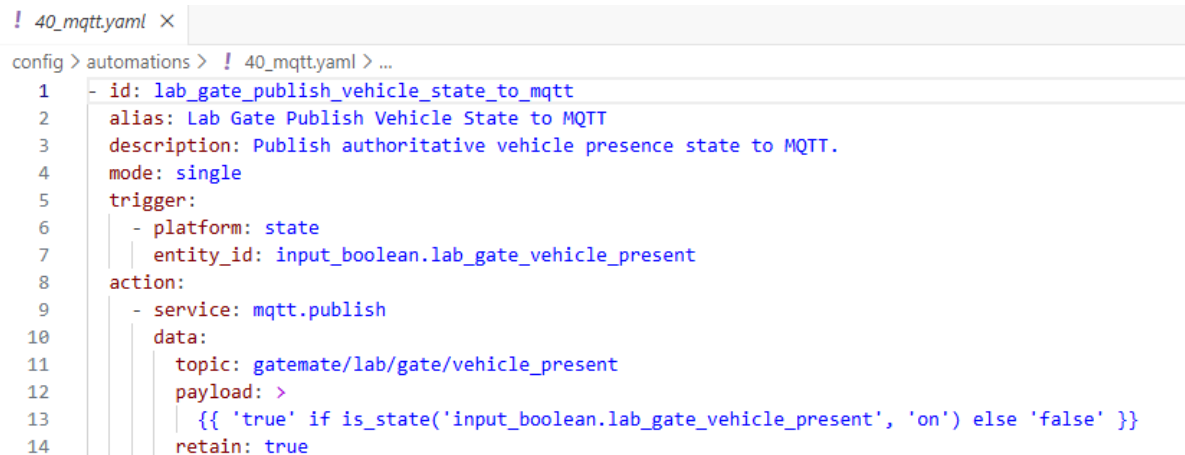


Figure 14 Diagnostic Dashboard, V1.0

8.2.3. MQTT State Publication

Sprint 4 implemented and validated MQTT-based state publishing to decouple presence detection from downstream consumers. The authoritative presence state (*input_boolean.lab_gate_vehicle_present*) is published to the topic *gatemate/lab/gate/vehicle_present* using retained messages. This is seen in Figure 15. This ensures that any subscriber joining the topic receives the current system state immediately, without requiring a new event to occur.



```
! 40_mqtt.yaml X
config > automations > ! 40_mqtt.yaml > ...
1 - id: lab_gate_publish_vehicle_state_to_mqtt
2   alias: Lab Gate Publish Vehicle State to MQTT
3   description: Publish authoritative vehicle presence state to MQTT.
4   mode: single
5   trigger:
6     - platform: state
7       entity_id: input_boolean.lab_gate_vehicle_present
8   action:
9     - service: mqtt.publish
10      data:
11        topic: gatemate/lab/gate/vehicle_present
12        payload: >
13          {{ 'true' if is_state('input_boolean.lab_gate_vehicle_present', 'on') else 'false' }}
14        retain: true
```

Figure 15 Code Extract Showing MQTT Publishing

Validation was performed using the built-in MQTT tools in HA. Publish/subscribe behaviour was verified by manually toggling the authoritative Boolean and confirming correct payload delivery (true / false). Retained message behaviour was confirmed by unsubscribing and re-subscribing to the topic, which resulted in immediate delivery of the last known state. Broker restart resilience was also tested by restarting the Mosquitto add-on and confirming that the retained state persisted and was correctly replayed to new subscribers. Some examples of this are shown in Figure 16.

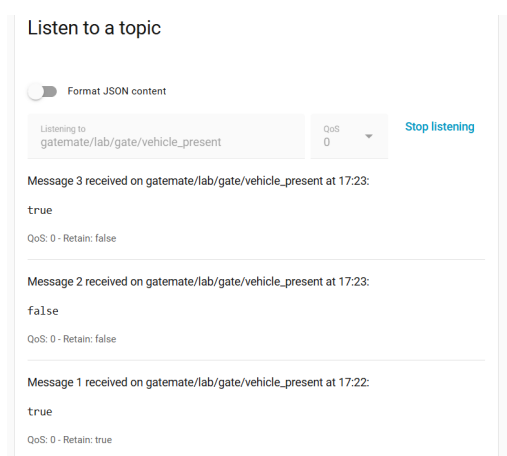


Figure 16 Built-in HA MQTT Listener demonstrating presence detection

An external MQTT client (mobile application) was used to demonstrate transport-layer decoupling. At this point, the demonstration was on the home-network only. The external client

subscribed to the presence topic without requiring access to the HA interface and received real-time state updates. This is demonstrated in Figure 17. This validates the architectural objective of separating detection logic from notification transport, enabling integration with non-HA consumers in future iterations.

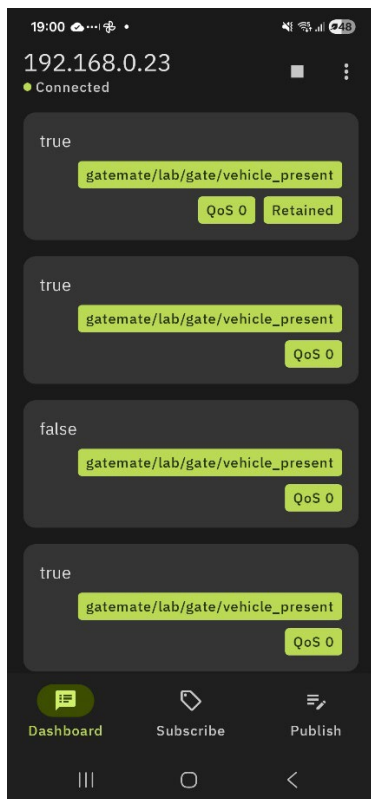


Figure 17 MQTT Client subscribed to GateMate presence alerts

The introduction of MQTT publishing establishes a clear architectural separation between the detection layer and downstream system consumers. By exposing the authoritative presence state as an event stream rather than embedding notification logic directly within automations, the system adopts an event-driven architecture. This allows additional consumers, such as external devices, dashboards or future integrations, to subscribe to the presence state without modifying the core detection logic. This is illustrated in Figure 18.

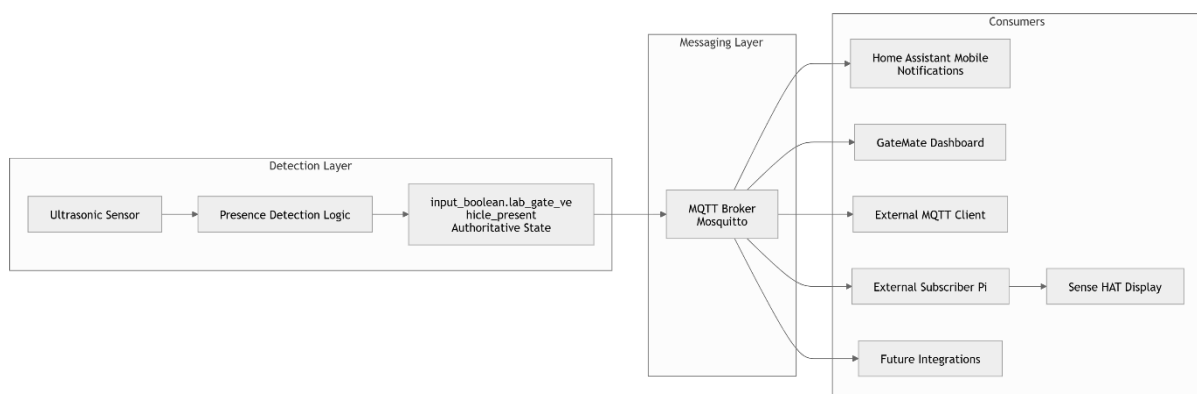


Figure 18 Event driven architecture demonstrating MQTT decoupling between detection logic and downstream consumers

8.2.4. MQTT Notifications beyond the Home Network (Stretch Goal)

To demonstrate architectural extensibility beyond the HA ecosystem, an external MQTT subscriber was implemented on a separate Raspberry Pi.

The subscriber connects to the Mosquitto broker using a dedicated service account and listens to the presence topic: `gatemate/lab/gate/vehicle_present`

When a “true” payload is received, the subscriber forwards the event to a Ntfy topic using a simple HTTP POST request. This is shown in Figure 19. The Ntfy service then delivers push notifications to any smartphone subscribed to that topic.

```
def ntfy_send(title: str, message: str) -> None:
    url = f"{NTFY_SERVER.rstrip('/')}/{NTFY_TOPIC}"
    headers = {"Title": title}
    resp = requests.post(url, data=message.encode("utf-8"), headers=headers, timeout=10)
    resp.raise_for_status()

def parse_presence(payload: str) -> bool | None:
    p = payload.strip().lower()
    if p in ["true", "on", "1", "present", "yes"]:
        return True
    if p in ["false", "off", "0", "clear", "no"]:
        return False
    return None
```

Figure 19 Code Extract Showing a portion of the python subscriber script

This allows gate presence alerts to be received without requiring access to the HA interface or direct MQTT connectivity. This is evidenced in Figure 20. The NTFY also gives alerts on any connected smartwatches, as seen in Figure 21. This demonstration shows that the system architecture supports integration with external systems while maintaining complete separation between detection logic and notification transport.

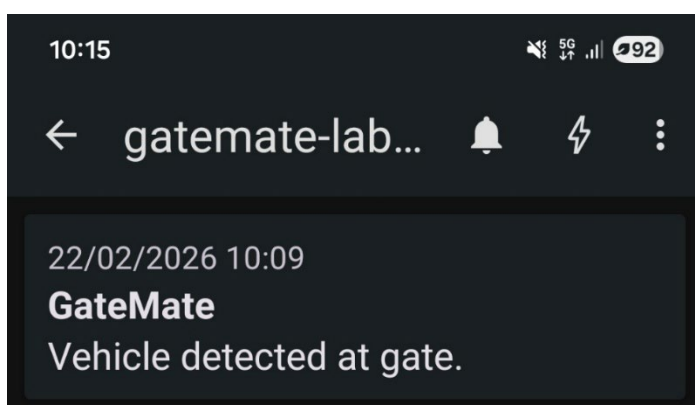


Figure 20 Phone notification on Ntfy app while connected to 5G only.

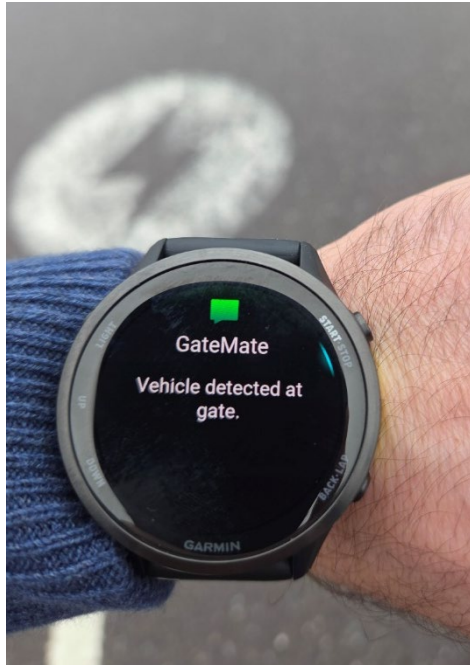


Figure 21 Alerts on a wearable device, via Ntfy app and MQTT

8.3. Evidence and Validation

System behaviour was validated using the HA history tools. This is shown in Figure 22 - Figure 24 demonstrate the authoritative presence state transitioning correctly between detected and cleared states in response to sensor input. These traces confirm that the notification layer responds correctly to changes in the authoritative presence state.

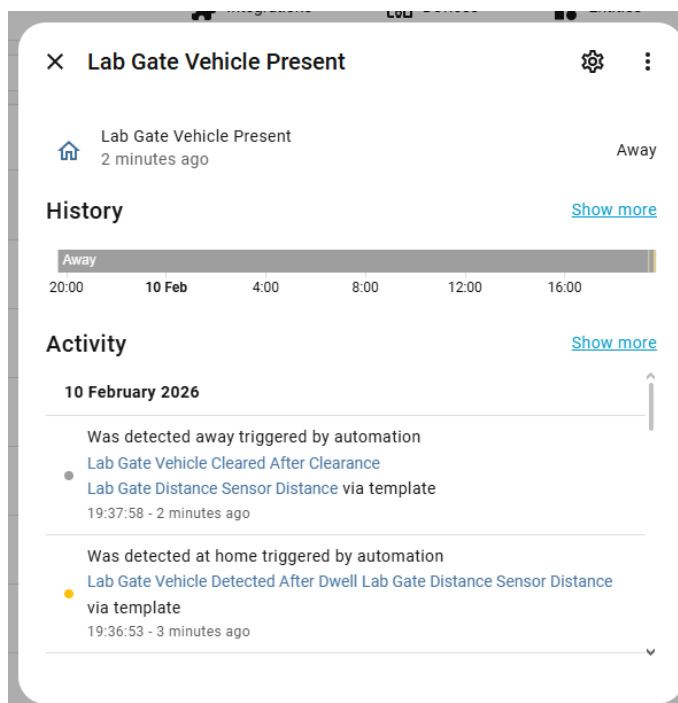


Figure 22 Presence Detection Helper History, toggling between states

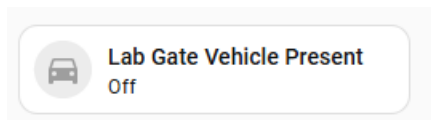


Figure 23 Presence Detection Toggled Off

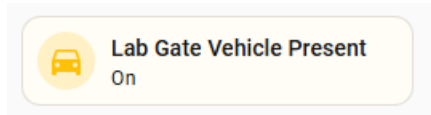


Figure 24 Presence Detection Toggled On

8.4. Outcome

Sprint 3 successfully extended the GateMate prototype from a local detection system into a complete notification platform, including support for:

- Operational dashboards
- Mobile push notifications
- MQTT-based state publishing
- External subscriber integrations

This demonstrates that the detection logic operates as a modular service within an event-driven architecture.

8.5. Limitations and Issues Encountered

The introduction of multiple notification channels increased system complexity, particularly in testing and coordination across platforms.

This highlighted the importance of maintaining strict separation between detection logic and notification delivery to preserve system clarity and manageability.

8.6. Implications for next Iteration

With detection logic, notifications and transport decoupling implemented, the next phase could focus on physical system integration and hardware deployment.

9. Sprint 4 – Hardware

Sprint 4 focused on integrating the sensing hardware into a portable deployment prototype.

The key objective was to establish a reliable communication path between the BLE distance sensor and the HA processing environment when installed at the gate location, which lies beyond the range of the native Bluetooth adapter.

9.1. Pre-Sprint Objectives

The objectives for this sprint are shown in Figure 25. These tasks aimed to transition the project from a purely software-based prototype to a physically deployable sensing system.

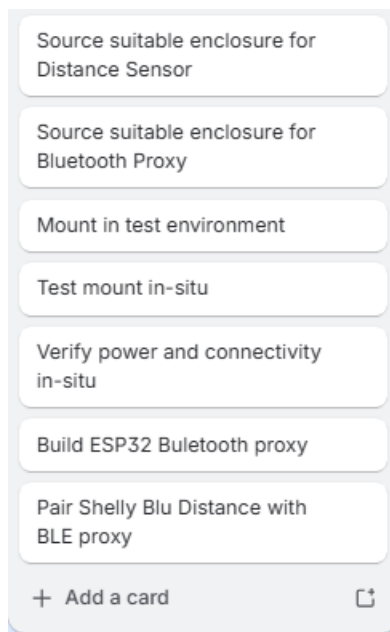


Figure 25 Sprint 4 Objectives (Trello)

9.2. Implementation and Technical Approach

9.2.1. ESP32 Bluetooth Proxy – Stretch Goal

Initial testing in Sprint 1 confirmed that the gate installation location was beyond the effective Bluetooth range of the Raspberry Pi hosting HA. To address this limitation, an ESP32-based Bluetooth proxy was implemented using ESPHome. The proxy scans for Bluetooth Low Energy advertisements from the Shelly Blu Distance sensor and forwards these packets to HA over the Wi-Fi network.

This was a tedious experience with slow flashing cycles, driver issues and malfunctioning cables. The documentation was also fragmented and hasn't been updated to reflect most recent HA UI changes. The proxy and both adapters are shown in Figure 26 and Figure 27 respectively.

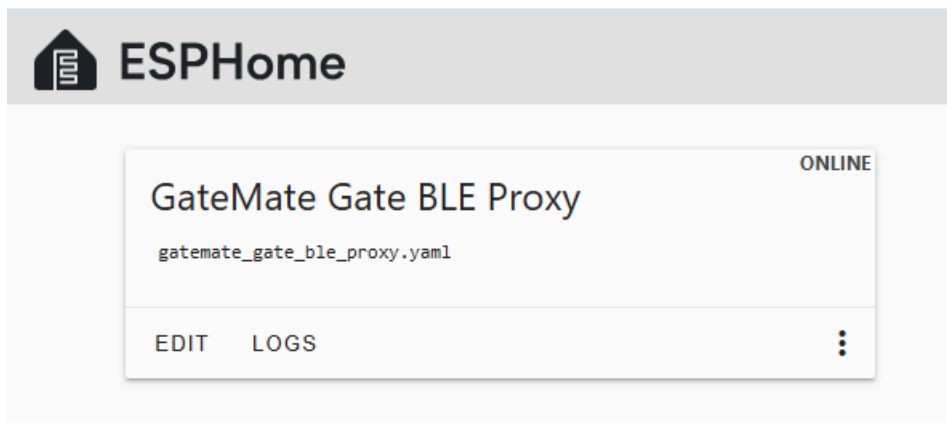


Figure 26 ESP32 Bluetooth Proxy, showing in ESPHome, within HA.

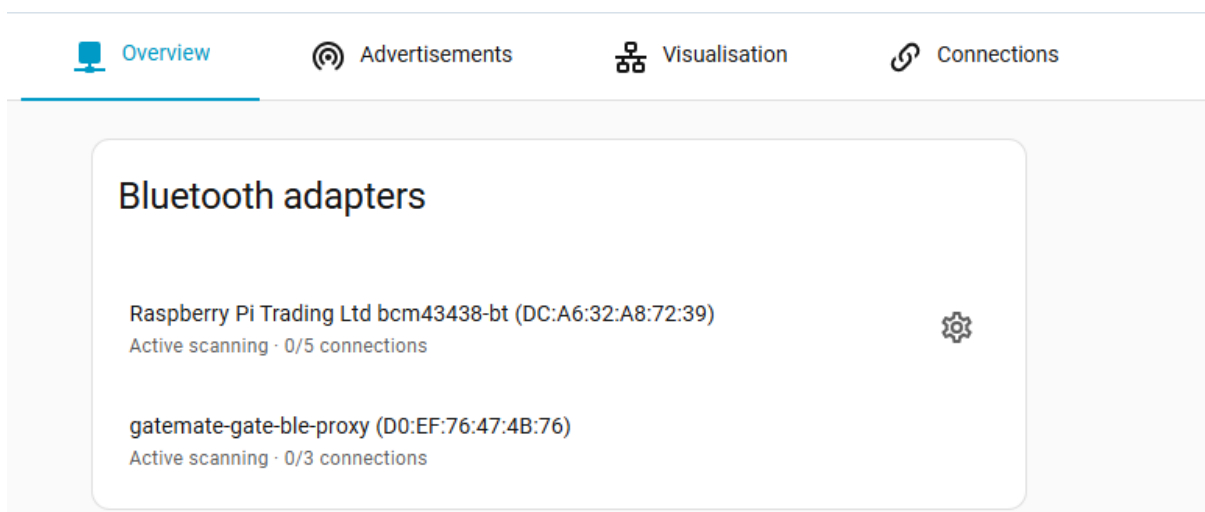


Figure 27 Built in Bluetooth adapter & new proxy both visible in HA.

Following successful deployment, HA began receiving BLE advertisements from the Shelly Blu Distance sensor via the proxy device. No modifications were required to the detection logic implemented in earlier sprints, demonstrating the benefit of the system's modular architecture. The communication pathway is illustrated in Figure 28.



Figure 28 BLE Direct Transport Path

9.2.2. Enclosure Fabrication and Sensor Mounting

To support outdoor deployment testing, the sensing hardware was installed in a compact weather-resistant enclosure. The enclosure was modified to allow the ultrasonic sensor to protrude through the front face while maintaining environmental protection for the internal electronics.

A hole was drilled into the enclosure lid to accommodate the threaded body of the sensor housing. The sensor was then secured using its integrated mounting nut with seals either side, ensuring stable alignment of the ultrasonic emitter with the detection area. This is shown in Figure 29.

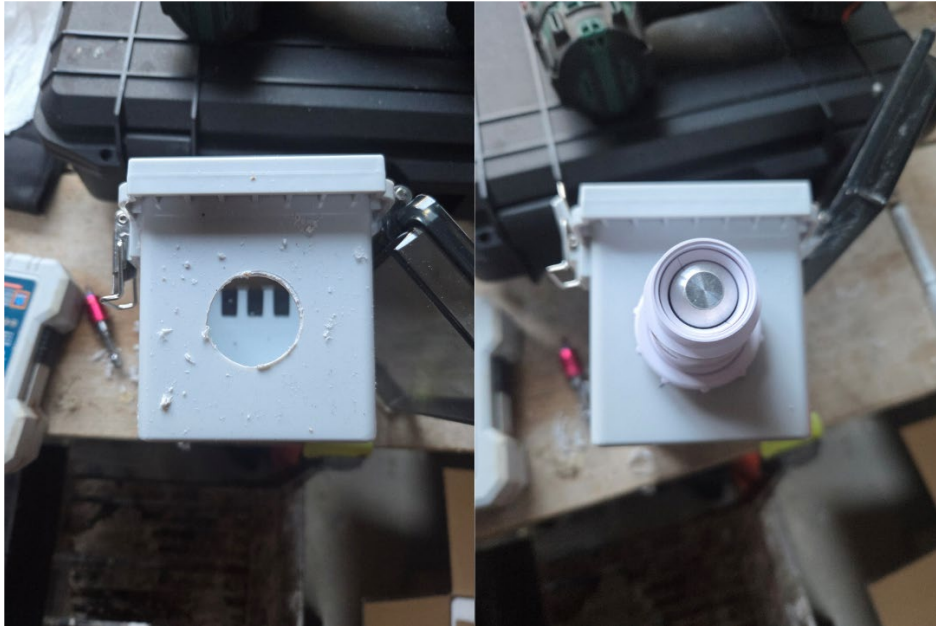


Figure 29 Enclosure adapted to take Ultrasonic Sensor

9.2.3. Portable Hardware Assembly

The prototype sensing node was assembled within a compact IP-rated enclosure to support portable deployment during testing. The ESP32 proxy (Figure 30) and the USB power bank (stretch goal) were also fitted in the same enclosure resulting in a fully sealed assembly. A small desiccant pack was also included to reduce internal humidity and protect the electronics during outdoor testing. The result is shown in Figure 31.



Figure 30 ESP32 Bluetooth Proxy in compact housing



Figure 31 GateMate hardware in sealed housing

9.2.4. Battery Operation (Stretch Goal)

As a stretch objective, the sensing node was designed to operate independently of fixed power infrastructure. This was achieved by powering the ESP32 Bluetooth proxy using a compact USB power bank housed within the enclosure.

Initially testing was problematic – the low power drawn by the ESP32 board was causing the power bank to automatically power off as it didn't register any load. This was overcome by reverting to an older, more basic power bank without built-in power management functions.

This configuration allows the sensing unit to function as a self-contained wireless sensing node, requiring only Wi-Fi connectivity to communicate with the HA host system. The approach removes the need for external cabling or nearby power outlets, simplifying deployment and allowing rapid repositioning of the device during testing.

In addition to improving deployment flexibility, the battery-powered design demonstrates how the sensing architecture could support fully wireless installations in environments where fixed power is unavailable.

The resultant hardware architecture is illustrated in Figure 32.

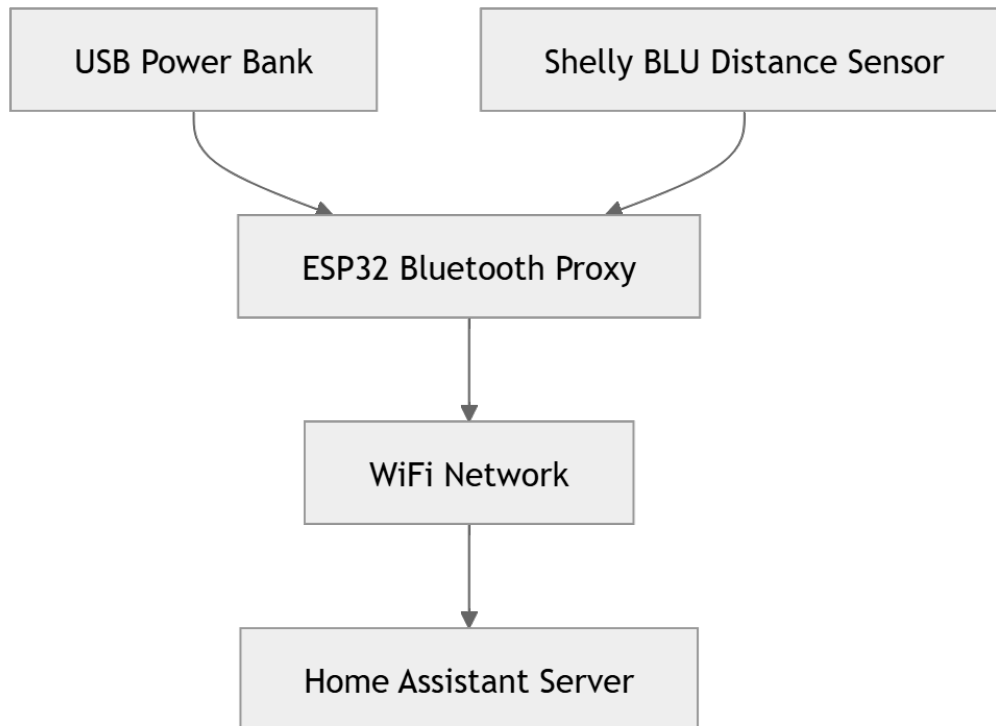


Figure 32 Hardware architecture of the portable sensing node

9.3. Evidence and Validation

Successful operation of the Bluetooth proxy was validated through the HA Bluetooth integration interface. Figure 26 and Figure 27 that both the built-in Bluetooth adapter and the newly deployed ESP32 proxy were recognised by HA. Sensor telemetry continued to be received after the sensing node was moved beyond the range of the host Bluetooth adapter.

Distance measurements received through the proxy were consistent with those previously observed during laboratory testing, confirming that the proxy successfully relayed BLE traffic without altering sensor behaviour.

9.4. Outcome

A portable sensing node was successfully developed, enabling deployment at the intended gate location.

The combination of a Bluetooth proxy and battery power allowed the system to operate independently of fixed infrastructure while maintaining reliable communication with HA.

9.5. Limitations and Issues Encountered

Implementation of the ESP32 proxy presented several practical challenges. Flashing the ESP32 device required multiple attempts due to driver compatibility issues and unreliable USB cables. Flashing cycles were slow and debugging was hindered by fragmented or outdated documentation within the ESPHome ecosystem.

The portable enclosure used during testing also represents a prototype rather than a final installation design.

9.6. Implications for next Iteration

With sensing and communication now functioning in a real-world environment, subsequent work focused on improving user interaction and system feedback through additional interfaces and integrations.

10. Sprint 5A – Backlog Work

This work took place outside of the neat sprint structure and is presented below as discrete work packages. The objectives are shown in Figure 33.

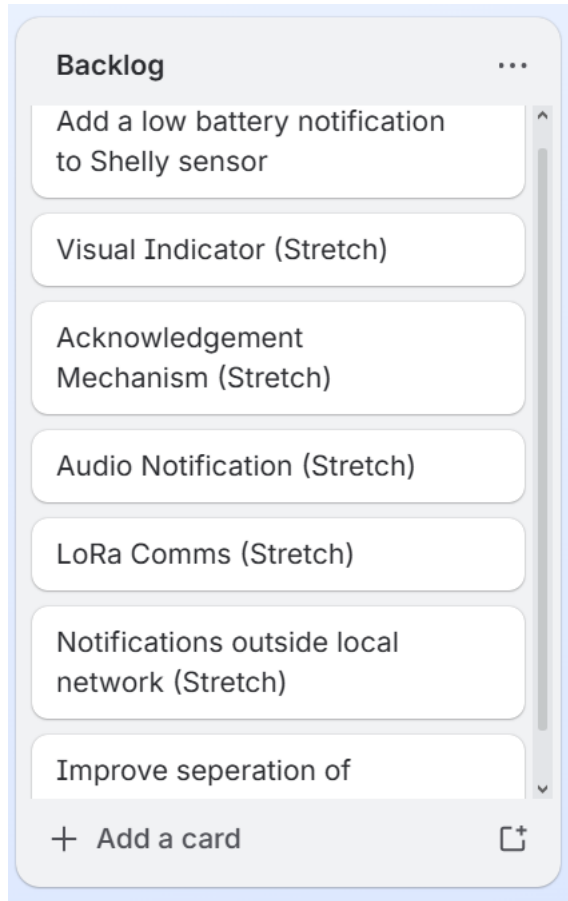


Figure 33 Backlog Work, Sprint 5 (Trello)

10.1. Low Battery Notification

A modular low-battery notification was implemented for the distance sensor. This approach can be extended to other devices in future. An extract of this code is shown in Figure 34.

```

! 50_battery.yaml X
config > automations > ! 50_battery.yaml > ...
1  - id: lab_gate_low_battery_alert
2    alias: "GateMate: Low battery alert (Lab Gate Distance Sensor)"
3    description: "Notify when Shelly Blu Distance battery is low, with dedupe and re-arm."
4    mode: single
5
6    trigger:
7      - platform: numeric_state
8        entity_id: sensor.lab_gate_distance_sensor_battery
9        below: 10
10
11    condition:
12      - condition: state
13        entity_id: input_boolean.lab_gate_battery_alert_sent
14        state: "off"
15
16    action:
17      - variables:
18        device_name: "Lab Gate Distance Sensor"
19        battery_entity: "sensor.lab_gate_distance_sensor_battery"
20        battery_percent: "{{ states(battery_entity) | int(0) }}"
21
22      - action: script.gatemate_notify_low_battery
23        data:
24          device_name: "{{ device_name }}"
25          battery_entity: "{{ battery_entity }}"
26          battery_percent: "{{ battery_percent }}"
27

```

Figure 34 Code Extract Showing a part of the low battery notification

10.2. Separation of concerns

As the project progressed, the single automations.yaml file became increasingly difficult to maintain.

To address this, the configuration was refactored using HA's "`!include_dir_merge_list`" command. Automations were split into multiple files within an automations/ directory, illustrated in Figure 35

```

! configuration.yaml 5 X
config > ! configuration.yaml > ...
8  # GateMate Lab modular includes
9  input_boolean: !include helpers/helpers_input_boolean.yaml
10 input_number: !include helpers/helpers_input_number.yaml
11 input_datetime: !include helpers/helpers_input_datetime.yaml
12 template: !include_dir_merge_list templates/
13

```

Figure 35 Refactoring Config file to improve readability etc

This refactoring improved traceability, improved readability during debugging, and reinforced the modular design principles defined during the RAMP phase.

10.3. Visual Notification

The external MQTT subscriber was extended to support a physical display using a Raspberry Pi Sense HAT.

This was implemented as a separate module, preserving separation of concerns. The display responds to the retained MQTT topic, allowing it to recover the correct state after restart without additional logic.

This demonstrates the extensibility of the system and the effectiveness of the decoupled architecture.

10.4. Outcome

Following the completion of this backlog work, most stretch goals have been achieved, and the system was now robust and functional. The system architecture is shown in Figure 36.

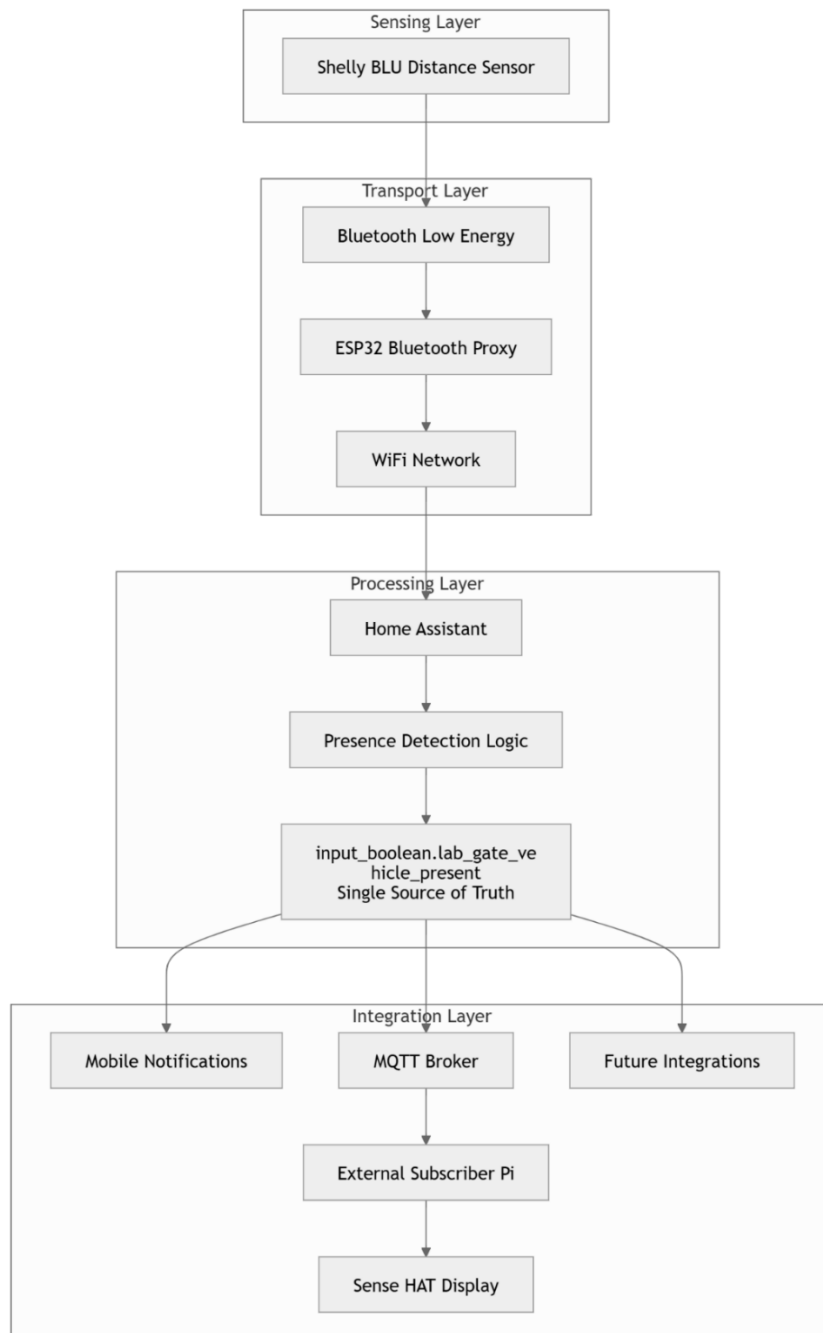


Figure 36 GateMate system architecture showing sensing, transport, processing, and integration layers

11. Sprint 5B - LoRa (Stretch Goal)

The objective was to address limitations observed in the existing BLE-based transport layer, particularly in relation to range and reliability. While functional, the BLE approach proved sensitive to environmental conditions and distance from the HA instance. This was treated as a mini sprint and added a significant degree of additionally complexity to the project.

11.1. Pre-Sprint Objectives

The objective was to add long range communication by utilising two Heltec Wi-Fi LoRa 32(V4), ESP32S3 boards.

11.2. Implementation and Technical Approach

There were shipping delays with this hardware due to Chinese New Year, followed by the United States' attacks on Iran and the consequent impact on airfreight. As a result, some of the coding was done in advance of the hardware arriving. The code used was informed by the RadioLib open source LoRa library (Gromes J. n.d.).

A LoRa-based communication layer was introduced using dedicated sender and receiver nodes. The sender node captured sensor readings and transmitted them over LoRa to a receiver node located within network range.

The Shelly BLU Distance sensor transmits BLE advertisements to a Heltec LoRa sender node. The sender extracts the relevant values and transmits them via LoRa to a corresponding Heltec receiver node. The receiver then publishes the values to the Mosquitto MQTT broker running on the HA instance. HA reads these MQTT topics as sensor entities, providing a second end-to-end communication path for the same sensor data. This additional pathway is shown in Figure 37.

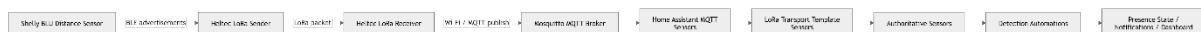


Figure 37 LoRa relay transport path

By maintaining MQTT as the integration layer, the system preserved a clear separation between transport and processing. This allowed the LoRa implementation to be introduced without impacting existing automation logic.

Additional handling was implemented to ensure message consistency and avoid duplication where multiple transport paths were present.

Testing was carried out at each stage to verify message integrity and delivery. The implementation was deliberately isolated from the core system during development to minimise disruption.

This approach ensured that the new transport layer could be introduced safely while maintaining system stability.

11.2.1. Dual Communication Architecture

Introducing a second transport path created a design challenge. If HA automations referenced BLE entities directly, a parallel set of automations would have been required to support the LoRa transport path. This would have created unnecessary duplication and tightly coupled the detection logic to a particular communication mechanism.

To avoid this, a transport abstraction layer was introduced. This layer separates the system into three distinct logical stages, further illustrated in Figure 38:

1. Transport layer – raw sensor entities originating from either BLE or MQTT.
2. Transport adapter layer – template sensors that normalise the LoRa MQTT data into standard HA entities.
3. Authoritative sensor layer – sensors that dynamically select the active transport source based on a configuration helper.

The authoritative sensors are controlled by the helper entity `input_select.lab_gate_transport_mode`, which allows the system to switch between BLE and LoRa transport paths. Crucially, the vehicle detection automations reference only the authoritative sensors rather than the underlying transport entities. This ensures that the detection logic remains completely independent of the communication mechanism used to deliver the sensor data.

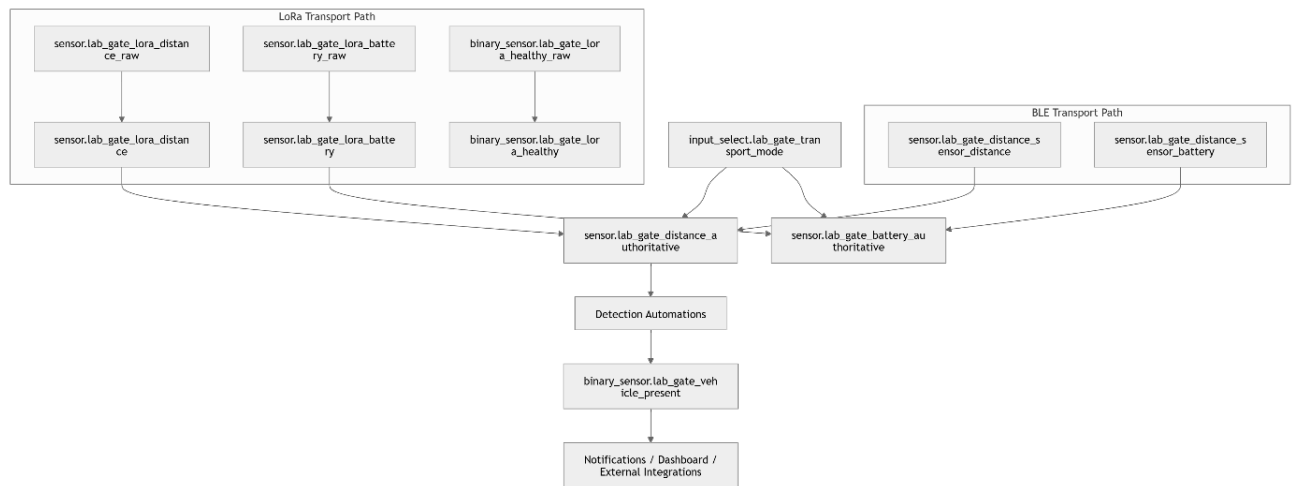


Figure 38 Authoritative sensor abstraction layer

The introduction of a dual communication architecture and a transport abstraction layer significantly improved the modularity and robustness of the GateMate system. The Wi-Fi path remains the original baseline implementation, while the LoRa path demonstrates extended communication range and architectural flexibility. By separating transport from logic, the system becomes easier to maintain, extend, and evaluate.

11.3. Evidence and Validation

The LoRa transport layer was validated through end-to-end testing of message transmission from sensor to MQTT broker. Reliable communication was achieved over distances of 100m+. greater distances are theoretically possible but haven't been tested.

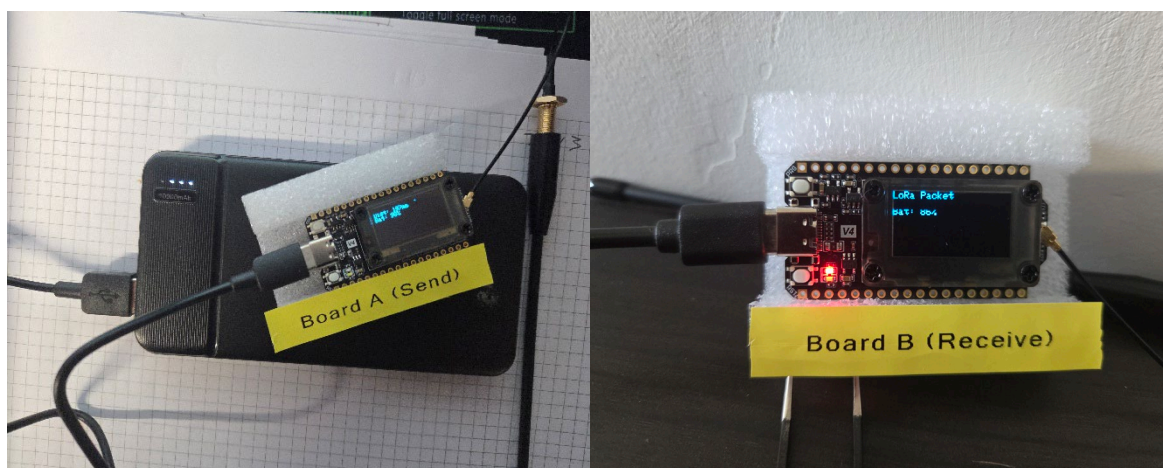


Figure 39 LoRa Boards sending and receiving information

Message delivery was consistent, with no observed dropouts during testing. Integration with MQTT was verified by confirming that downstream components received and processed messages identically to those delivered via BLE.

In the current implementation, both systems operate in tandem, with priority given to BLE if it is available. The updated hardware is shown in Figure 40



Figure 40 Updated hardware including LoRa antenna

11.4. Outcomes

The introduction of LoRa significantly improved the reliability of the transport layer. Communication was maintained over distances where BLE performance had degraded, resulting in more consistent message delivery.

The system architecture accommodated this change without modification to the processing layer, confirming the effectiveness of the MQTT-based abstraction. This demonstrated that transport mechanisms could be extended without impacting core system logic.

11.5. Limitations and Issues Encountered

The LoRa implementation increased system complexity, requiring additional hardware and configuration.

Lower data rates limit suitability for high-frequency data transmission, although this was not a constraint in this use case.

Battery optimisation was not addressed in the current implementation.

The BLE communication between the distance sensor and the Lo-Ra sender board is effectively hardcoded. This would be something which could be improved upon in future iterations.

11.6. Implications for next iteration

The addition of LoRa enables deployment in low-connectivity environments and improves system resilience.

However, it introduces trade-offs between simplicity and robustness that must be considered in future implementations.

12. Sprint 6 - Refinement

12.1. Pre-Sprint Objectives

The objective of Sprint 6 was to refine and stabilise the GateMate system following the completion of core functionality, see Figure 41.

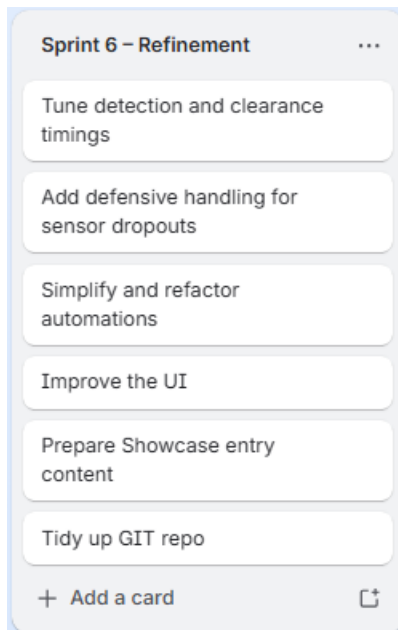


Figure 41 Sprint 6 Objectives (Trello)

12.2. Implementation and Technical Approach

12.2.1. Automation Refactoring

HA automations were refactored to reduce complexity and remove duplicated logic. Naming conventions were standardised, resulting in a clearer and more maintainable configuration.

12.2.2. Detection Tuning and Defensive Handling

Detection and clearance timings were adjusted to reduce false positives and ensure stable state changes. Defensive handling was introduced to manage sensor dropouts, maintaining the last valid state during short interruptions.

12.2.3. User Interface Improvements

The interface was simplified by replacing ambiguous labels such as “Home” and “Away” with states that directly reflect vehicle presence. This reduced user interpretation and improved clarity.

12.2.4. Repository Organisation

The Git repository was cleaned and reorganised to improve structure and traceability. Files were modularised and commit history aligned to logical development steps. Folders were removed where possible and added to .Gitignore.

12.2.5. ReadMe Refinements

A full audit of the various ReadMe files was completed with a more complete set of instructions included for the main hardware as well as the associated integrations.

12.3. Approach Taken

Sprint 6 focused on improving system reliability through targeted refinement rather than introducing new features. Changes were applied incrementally to isolate their impact and avoid destabilising the system.

Refactoring was prioritised to simplify the automation layer before further adjustments were made.

User interface changes were implemented to align system feedback with underlying logic, reducing ambiguity. Repository clean-up was carried out alongside these changes to maintain a clear and traceable development history.

The dual transport layer switching was tested extensively.

12.4. Evidence and Validation

System behaviour was validated through repeated real-world testing of vehicle detection scenarios. The time-threshold logic consistently filtered short-duration events, reducing false positives and improving stability.

Sensor dropouts were simulated through temporary signal interruption. The system maintained the last valid state during these events, confirming the effectiveness of the defensive handling approach.

User interface changes were validated through direct interaction, confirming that updated labels more accurately reflected system state and reduced ambiguity. This is illustrated in Figure 42 where the key detection question is clearly answered with a “Yes” or a “No”.

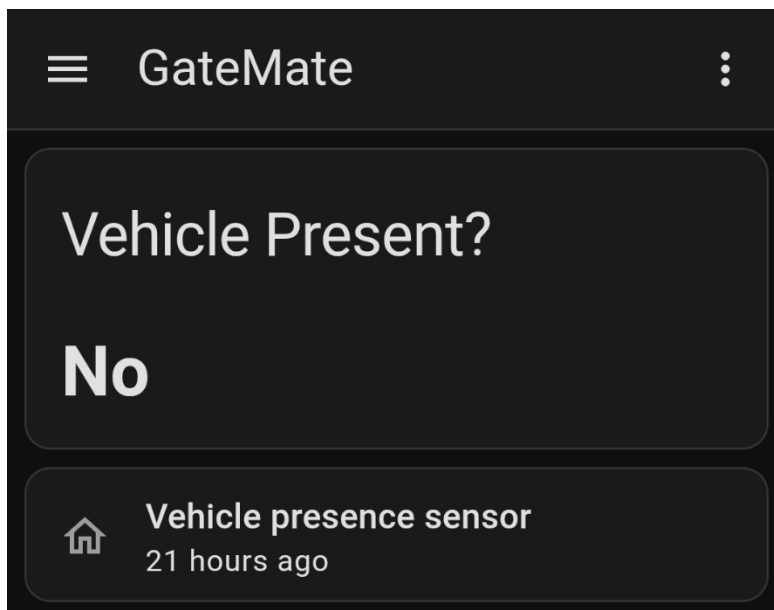


Figure 42 UI Improvements to clearly convey the relevant information

Repository improvements were verified through inspection of commit history and file structure, demonstrating clearer organisation and logical progression of changes.

The BLE to LoRa dual transport layer was tested extensively. The Diagnostic Dashboard is shown below in Figure 43. This illustrates current and correct distances in the BLE section and older incorrect measurements in the LoRa section (LoRa was physically disconnected). This validates the behaviours as the BLE switch was active ensuring that incorrect measurement was ignored.

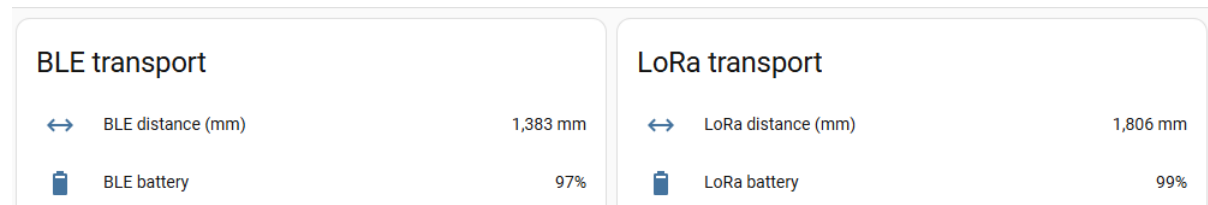


Figure 43 Diagnostic Dashboard showing different readings when LoRa is unavailable (LoRa reading is old)

12.5. Outcome

Sprint 5 resulted in a more stable and maintainable system. Defensive handling improved resilience to temporary sensor dropouts without introducing additional complexity.

Refactoring simplified the automation layer, reducing the risk of conflicting logic and making the system easier to modify. User interface changes improved clarity by aligning displayed states directly with vehicle presence.

Repository clean-up improved structure and traceability, supporting reproducibility and clearer version control.

13. Reflections

Before I reflect on the project specifically, I want to reflect on the course in a broader sense. I started this course around the same time my daughter started primary school in the local Gaelscoil. We were both learning a new language. I began with a “pigeon English” level of understanding, while she started with no real knowledge of the language.

For a long time, my daughter was making progress at a faster pace than I was, unhindered by any sense that the task should be difficult. In recent months I’ve noticed my own ability has improved as I completed this project. I found I can parse documents more efficiently and read code that would have previously been incomprehensible.

As with any language, fluency develops through exposure and immersion. While my daughter will continue that immersion for many more years, my exposure will likely reduce soon. Regardless I’m more than satisfied with the new knowledge I’ve attained and I look forward to applying it in my own career.

Project specifics reflections are considered in the following sections.

13.1. What I Learned

Modular architecture becomes essential as system complexity increases. Early development prioritised rapid implementation, the addition of MQTT, notifications, and monitoring required a more structured approach.

Separating sensing, transport, and processing proved critical. By abstracting the transport layer, the system supports both BLE and LoRa without altering core detection logic, significantly improving extensibility and validating earlier architectural decisions. Extra time spent in the research phase identified this is a likely future step allowing me to plan accordingly.

The project demonstrated that Home Assistant can act as a processing and state management platform, not just a UI layer. Real-world hardware behaviour required iterative testing, reinforcing the need for validation over assumptions.

Finally, the project highlighted that maintainability, clarity, and traceability are core aspects of system design, particularly as systems evolve beyond simple prototypes.

13.2. What I Achieved

The project delivered a fully working, end-to-end vehicle presence detection system integrating sensing hardware, wireless communication, MQTT, and Home Assistant automation.

A layered architecture was implemented, separating sensing, transport, processing, and integration concerns, allowing the system to remain stable as functionality expanded. The project integrates multiple technologies, including BLE, LoRa, MQTT, and external notification services.

A key achievement was the implementation of a transport-agnostic design, enabling both Wi-Fi and LoRa communication paths. The addition of LoRa as a stretch goal increased technical scope significantly and demonstrated suitability for low-connectivity environments.

13.3. Problems Encountered and Solutions

Several practical challenges arose during development.

Initial setup issues included Git configuration within Home Assistant and ESP32 flashing, resolved eventually through persistent troubleshooting.

Sensor noise and inconsistent readings were addressed using dwell-time filtering, ensuring only sustained readings triggered presence detection.

As functionality increased, automation configuration became difficult to manage. Refactoring into a modular YAML structure aligned with the system architecture improved clarity, reduced debugging risk, and enhanced traceability.

These challenges reinforced the importance of iterative development, structured debugging, and designing for maintainability from an early stage.

14. Conclusion

The project delivers on the MVP and many of the stretch goals. The prototype has been working in the real world for many weeks now and can therefore be considered a success.

15. Bibliography

Agile Business Consortium (2022). *Chapter 10: MoSCoW Prioritisation*. [online] [www.agilebusiness.org](https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html). Available at: <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html> [Accessed 16 Jan. 2026].

Home, A. (n.d.). *Documentation*. [online] Home Assistant. Available at: <https://www.home-assistant.io/docs/> [Accessed 17 Jan. 2026].

Gromes, J. (n.d.). *RadioLib Universal wireless communication library for embedded devices*. Available at <https://github.com/jgromes/RadioLib/tree/master> [Accessed 18 Feb. 2026]

Rehkopf, M. (2019). *User Stories*. [online] Atlassian. Available at: <https://www.atlassian.com/agile/project-management/user-stories> [Accessed 16 Jan. 2026].

16. Appendices

16.1. Appendix – Trello Boards

GateMate

mq

14 days left

14 days left

q. Search

Create

14 days left

14 days left

14 days left

14 days left

Backlog

...

Acknowledgement Mechanism (Stretch)

Visual Indicator (Stretch)

Audio Notification (Stretch)

LoRa Comms (Stretch)

Notifications beyond local network (Stretch)

+ Add a card

Sprint 1 – RAMP Completion

...

Finalise methodology section

Finalise system model

Finalise data flow model

Finalise state model

Lock MVP scope

Set up standalone Home Assistant environment

Create Trello sprint structure

Prepare Interim Report for submission

+ Add a card

Sprint 2 – BLE Integration

...

Pair Shelly Blu Distance with BLE proxy

Verify sensor entities in Home Assistant

Measure BLE update frequency

Test BLE range at gate location

Document BLE reliability and constraints

+ Add a card

Sprint 3 – Presence Logic

...

Define distance threshold parameters

Implement dwell-time filtering logic

Implement clearance-time logic

Create presence state helpers

Validate state transitions with live data

+ Add a card

Sprint 4 – Notifications

...

Define presence detected automation

Define presence cleared automation

Configure mobile push notifications

Create GateMate dashboard view

Validate repeated arrival/departure behaviour

+ Add a card

Sprint 5 – Refinement

...

Tune detection and clearance timings

Add defensive handling for sensor dropouts

Simplify and refactor automations

Capture screenshots for report

Prepare Showcase entry content

+ Add a card

Sprint 6 – Code Freeze

...

Final regression testing at gate location

Finalise Home Assistant configuration

Clean and document Git repository

Declare code complete

Consolidate final report content

Prepare project demo flow

+ Add a card

46

16.2. Appendix - Glossary of Terms

Add-ons – HA Specific. These are isolated Docker containers that run third-party software (e.g., MQTT brokers) alongside HA to extend the system's capabilities.

BLE – Bluetooth Low Energy

Dashboards – HA Specific. these are the primary UX interface.

Entities – HA Specific. these are the basic building blocks that represent individual data points or controls, such as a temperature sensor reading. Each device may have multiple entities.

ESP32 – is a low-cost System on a Chip microcontroller with integrated wi-fi and Bluetooth. It is popular for IOT projects.

GUI – Graphical User interface

Helpers – HA Specific. these are user-created virtual entities. They are used to store custom data, parameters, perform calculations, or act as toggles to assist in complex automations.

HA – Home Assistant

Integrations – HA Specific. these are software modules that bridge HA with external devices (e.g., Philips Hue) and services (e.g., weather providers)

LoRa – Long Range Radio – LoRa is a low-power, wide-area wireless radio communication technology. It enables battery-operated devices to transmit small amounts of data over long distances—typically greater than 1km.

RPi – Raspberry Pi

TTS – Text to Speech

UI – User Interface

YAML (Yet another markup language) – this is the language used for the platform's underlying configuration files and advanced automation scripting.

16.3. Appendix – Use of AI

Chat GPT was used in for some specific elements of the project. AI has been used to edit sections of this submission for brevity, and academic tone as well as assuring a consistent structure. The technical content, system design, construction implantation, testing, analysis, effort and essence remain very much the author's own.

16.3.1. Creating mermaid diagrams

Chat GPT prompts were used to generate instructions for mermaid.ai to generate flowcharts used within this report. The outputs, below, were pasted into mermaid.ai to generate the charts.

“flowchart LR

```
subgraph External_Environment
```

```
  V[Vehicle or Visitor]
```

```
  U[Users]
```

```
end
```

```
subgraph GateMate_System
```

```
  S[Sensing Layer - Ultrasonic Sensor]
```

```
  C[Communication Layer - BLE or WiFi]
```

```
  P[Processing and State Logic]
```

```
  A[Automation and Integration - Home Assistant]
```

```
end
```

```
subgraph Interfaces
```

```
  D[Home Assistant Dashboard]
```

```
  N[Notifications - Mobile or Local]
```

```
end
```

```
V --> S
```

```
S --> C
```

```
C --> P
```

```
P --> A
```

```
A --> D
```

```
A --> N
```

```
D --> U
```


N --> U”

“sequenceDiagram

participant Sensor as Ultrasonic Sensor

participant Comm as BLE or WiFi Transport

participant HA as Home Assistant

participant Logic as Dwell Time Logic

participant Auto as Automation Engine

participant User as User

Sensor->>Comm: Distance reading

Comm->>HA: Sensor update

HA->>Logic: Evaluate distance and duration

Logic-->>HA: Presence state update

HA->>Auto: Trigger automation

Auto->>User: Notification or indicator”

stateDiagram

[*] --> Idle

Idle --> Presence_Detected: distance in range for dwell_time

Presence_Detected --> Presence_Cleared: distance out of range for clearance_time

Presence_Cleared --> Idle: reset after clear event

16.3.2. Formatting of readme md files.

The various readme files were converted into Markdown (.md) files and formatted using Chat GPT.

16.3.3. Logo design

The GateMate Logo was generated in ChatGPT.

